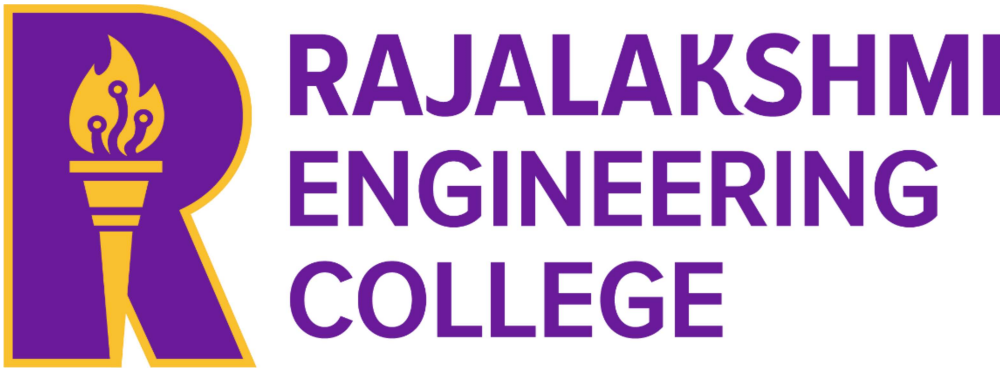
RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution)

RAJALAKSHMI NAGAR, THANDALAM- 602 105



CS23532 – COMPUTER NETWORKS

LABORATORY RECORD NOTEBOOK

NAME : _____

YEAR/SEMESTER : _____

BRANCH/SECTION : _____

REGISTER NO : _____

COLLEGE ROLL NO : _____

ACADEMIC YEAR : _____

RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution)

RAJALAKSHMI NAGAR, THANDALAM- 602 105

BONAFIDE CERTIFICATE

NAME: …… **BRANCH/SECTION:** ……

ACADEMIC YEAR: 20….. -20….. **SEMESTER:** ……..

REGISTER NO:

Certified that this is a Bonafide record of work done by the above student in the **CS23532 – Computer Networks** during the year 20 - 20

Signature of Faculty In-charge

Submitted for the Practical Examination Held on: ……..

Internal Examiner **External Examiner**

INDEX

EX.NO	DATE	NAME OF THE EXPERIMENT	GITHUB QR CODE	STAFF SIGN
1		Basic Networking Commands in Linux and Windows Operating Systems		
2		Learning and Assignment of IP Address manually to computers		
3		Study of different types of Network Cables and Crimping of cable with RJ45 connector		
4		Implement packet sniffing using raw sockets in python		
5		Develop a customized ping command to test the server connectivity		

6		Building anonymous FTP Scanner using ftplib module		
7		Develop a simple calculator using XMLRPC		
8		Develop a program to create reverse shell using TCP sockets		
9		Design a simple topology and configure with one router, two switches and PCs using Cisco Packet Tracer		
10		Customize Switch with Network Modules using Cisco Packet Tracer		
11		Examine Network Address Translation (NAT) using Cisco Packet Tracer		
12		Nmap to discover live hosts using ARP scan, ICMP scan, and TCP/UDP ping scan in TryHackMe Platform		
13		Demonstrate network forensics using PcapXray tool		

14		To capture, save, and analyze network traffic on TCP/UDP/IP/HTTP/ARP/DHCP/ICMP/DNS using Wireshark Tool		
15		To Analyze the different types of servers using Webalizer tool		

STUDY OF VARIOUS COMMANDS USED IN LINUX

EXP NO: 1

DATE : 17/07/2026.

AIM : TO study about various networking commands on Linux operating system.

BASIC LINUX COMMANDS:

pwd

Description: prints the current working directory.

Output:

$ pwd

/home/liveuser

mkdir

Description: creates a new directory

Output:

$mkdir fedora

$ ls.

Desktop Documents Downloads.

fedora

cd

Description: changes the current directory.

Output:

$ cd fedora

$ pwd

/home/liveuser/fedora

touch filename.txt

Description: Creates an empty file.

Output:

$ touch example.txt

$ ls.

example.txt

echo "Hi" > filename.txt

Description: write text into a file.

Output:

$ echo "Hi" > example.txt

Cat filename.txt

Description: Displays the contents of a file.

Output:

$ cat example.txt

Hi

grep

Description: searches for a specific pattern in a file.

Output:

$ grep "H" example.txt

Hi

wc

Description: Displays the number of lines, words, and byt in a file.

Output:

$ wc example.txt

1 1 3 example.txt

wc-l

Description: Counts the number of lines in a file.

Output:

$ wc -l example.txt

1 example.txt

wc -w

Description: Counts the number of words in a file.

Output:

$ wc -w example.txt

1 example.txt

wc -c

Description: Count the number of bytes in a file.

Output:

$ wc -c example.txt

3 example.txt

wc -m

Description: Count the number of characters in a file.

Output:

$ wc -m example.txt

3 example.txt

wc -L

Description: Displays the length of the longest line in a file.

Output:

$ wc -L example.txt

2 example.txt

Basic Networking commands

hostname

Description: Displays the system's hostname.

Output:

$ hostname

local host-live

Name

Description: Displays the routing table.

Output:

$ route

Kernel IP routing table

netstat

Description: Shows network connections and routing information.

Output:

$netstat

Active Internet connections.

IP address show

Description: Displays IP address and network interface details.

output:

$ ip address show

1: IO:

2: enp0s3:

nslookup

Description: Finds the IP address of a domain.

output:

$ nslookup www.google.com.

Name: www.google.com.

Address: 142.250.xxx.xxx

ifconfig.

Description: Displays network interface configuration.

Output:

$ ifconfig

enp0s3

inet 10.0.2.15.

Ping

Description: Tests connectivity to a host.

output:

$ ping www.google.com;

64 bytes from ...

arp

Description: Displays the ARP cache.

output:

$ arp -a

-gateway (10.0.2.2)

traceroute.

Description: Displays the route taken by packets to reach a destination.

output:

$ traceroute www.google.com:

1: -gateway

2:

RESULT: 17/7/26

Thus the various networking commands on Linux operating system is successfully verified and executed.

EXP NO: 02 DATE: 23/07/2026

LEARNING AND ASSIGNMENT OF IP ADDRESS MANUALLY TO COMPUTERS.

AIM:

To learn IP addressing and manually assign an Ip address to a computer. To verify the assigned IP address using Linux networking Commands.

THEORY:

An IP address is a unique numerical address assigned to every device connected to a network It usables communication between devices over a network. An IPV4 address consists of four octets separated by dots (Eg: 192.168.1.10).

An Ip address can be assigned either manually (Static Ip) or automatically (Dynamic IP) using DHCP. A manually assigned Ip address remain unless changed by the user Along with the Ip address, parameter such as the subnet Mask, Default Gateway, and DNS server are also configured for proper network communication.

In Linux, the IP Command is used to displ and manage IP address, network interfaces, routing tables and neighbouring devices It is one of the most Commonly used Commands for network Configuration and troubleshooting.

Commands and sample outputs.

1) Display all IP address.

Command: ip address show.

Sample output:

```
2: ens160:
   inet 172.16.211.128/24
   inet6 fe80::20c:29ff::fe75:9927/64.
```

Observation:

Displays the IPv4 and IPv6 addresses of all networks.

2) Display a specific network interface:

Command: ip addr show dev ens160.

Sample output:

```
2: ens160:
   inet 172.16.211.128/24.
```

3) Display Routing Table:

Command: ip route show

Sample output: default via 172.16.211.2 dev ens160

172.16.211.0/24 dev ens160.

observation: Displays the routing table and the default gateway.

4) Display Neighbour (ARP) Table:

Command: ip neigh show

sample output: 172.16.211.1 REACHABLE

172.16.211.254 STABLE.

Observation: Displays the neighbouring devices and their MAC address.

RESULT: The IP address was manually assigned and verified successfully using Linux networking commands.

EXP NO: 03 DATE: 23/07/2026

STUDY OF DIFFERENT TYPES OF NETWORK CABLES AND CRIMPING OF CABLE WITH RJ45 CONNECTOR

AIM:

To study different types of network cables and understand the crimping process using an RJ45 Connector.

CONCEPT:

Network Cables are used to connect computers and other networking devices for data communication. The most commonly used cable in LAN is the Unshielded Twisted pair (UTP) cable. An RJ45 Connection is attached to the ends of the cable to establish a network connection. The wires are arranged according to the 568A or T568B wiring standard before crimping, and the completed cable is tested to ensure proper connectivity.

Types of Network Cables.

Cable Type	Description
1) UTP (Unshielded Twisted pair)	Commonly used for Ethernet LAN's and office networks.
2) STP (Shielded Twisted pair)	Used in area with high electrical or electromagnetic interference.
3) Coaxial Cable	Used for Cable television, broadband internet, and CCTV systems.
4) Optical Fibre Cable	used for high-speed, high Bandwidth and long distance Communication.

CRIMPING PROCEDURE:

1) Cut the UTP Cable to the required length.
2) Strip about 2-3 cm of the outer insulation.
3) Untwist and straighten the wires.
4) Arrange the wires according to the T568B standard.
5) Trim the wire ends evenly.
6) Insert the wires into the RJ45 connector.
7) Crimp the connector using a crimping tool.
8) Test the Cable using a LAN cable tester.

OBSERVATION:

* The wires were arranged according to the T568B standard.
* The RJ45 Connector was crimped successfully.
* The Cable tester indicated proper continuity.
* The Cable was ready for network communication.

RESULT: The different types of network cables were studied, and the UTP cable was successfully terminated using and RJ45 Connector and tested for proper connectivity.

EXP NO: 5 DATE: 24/07/2026

DEVELOP A CUSTOMIZED PING COMMANDS TO TEST THE SERVER CONNECTIVITY

AIM: To develop and use customized ping commands to test the connectivity, response time, and packet transmission between a client system and a server.

USAGE OF PING:

* To check whether a remote host or server is reachable.
* To verify network connectivity between two devices.
* To measure the response time (latency) of a network.
* To detect packet loss during data transmission.
* To troubleshoot network connection problems.
* To test the availability of websites, routers, or servers.

PURPOSE OF PING:

* To ensure that a network device is active and responding.
* To identify connectivity issues in a network.

* TO evaluate the performance of a network connection.
* TO help diagnose delays or communication failures.
* TO verify that the IP address or domain name is accessible.
* TO assist network administrators in monitoring and maintaining network health.

PING CONNECTION ESTABLISHMENT:

* The User enters the ping command with the destination IP address or domain name.
* The computer sends an ICMP Echo Request packet to the destination device.
* The destination device responds with an ICMP Echo Reply if it is reachable.
* The system displays the response time and packet status, confirming whether the connection is established successfully.

WINDOWS PING COMMANDS:

1. **Basic Ping**

Command => ping google.com

Output => Reply from 142.250.193.14: bytes=32 time=18ms TTL=118.

2. **Ping 5 Times**

Command => ping -n 5 google.com

output => Packets: sent=5, Received=5, Lost=0 (0% loss).

3. **Continuous Ping**

Command => ping -t google.com

Output => Reply from 142.250.193.14: bytes=32 time=17ms TTL=118.

Reply from 142.250.193.14: bytes=32 time=18ms TTL=118.

4. **Ping With Large Packet**

Command => ping -l 1000 google.com

Output => Reply from 142.250.193.14: bytes=1000 time=20ms TTL=118.

5. **Ping Loopback Address**

Command => ping 127.0.0.1

Output => Reply from 127.0.0.1: bytes=32 time<1ms TTL=128.

6. Ping Google DNS

Command => ping 8.8.8.8.

Output => Reply from 8.8.8.8: bytes = 32 time = 25ms
TTL = 118:

7. Resolve Host Name

Command => ping -a 8.8.8.8.

Output => Pinging dns.google [8.8.8.8] with 32 bytes
of data:

8. Set Time out

Command => ping -w 1000 google.com.

Output => Reply from 142.250.193.14: bytes = 32 time = 18ms
TTL = 118.

9. Force Ipv4 Ping

Command => Ping -4 google.com

Output => Pinging google.com [142.250.193.14] with
32 bytes of data:

Reply from 142.250.193.14: bytes = 32
time = 18ms TTL = 118.

10. Prevent Packet Fragmentation:

Command => ping -f -l 1472 8.8.8.8

Output => Reply from 8.8.8.8: bytes =1472 time=24ms TTL=118.

RESULT: The ping commands were executed successfully, and the server connectivity was verified with successful replies and no packet loss.

EXP NO : 4. DATE : 06/08/2024

IMPLEMENT PACKET SNIFFING USING RAW SOCKETS IN PYTHON

AIM : To implement packet sniffing using raw sockets in Python and capture network packets passing through the network interface.

OBJECTIVE : To understand how raw sockets work and how network packets can be captured and analyzed using python programming.

THEORY :

* Packet sniffing is the process of monitoring and capturing data packets traveling across a network.
* A packet sniffer intercepts packets and analyzes their contents.
* Raw sockets provide direct access to lower-level network protocols, allowing applications to capture packets before they are processed by the operating system.
* In python, the socket module can be used to create raw sockets for packet capture.
* This technique is useful for network monitoring, troubleshooting, and security analysis.

ALGORITHM:

1. Import the required socket module.
2. Create a raw socket.
3. Bind the socket to the network interface.
4. Receive packets using the recvfrom() method.
5. Display the captured packet information.
6. Repeat the process continuously to monitor network traffic.

PROGRAM:

```
import socket
sniffer = socket.socket(socket.AF_INET, socket.SOCK_RAW,
          socket.IPPROTO_IP)
host = socket.gethostbyname(socket.gethostname())
sniffer.bind((host,0))
sniffer.setsockopt(socket.IPPROTO_IP,socket.IP_HDRINCL,1)
while True:
    raw_data, addr = sniffer.recvfrom(65535)
    print("Packet received from:", addr)
    print(raw_data)
```

OUTPUT:

```
Packet received from:
('192.168.1.5', 0)
b'E\x00\x00 <...'

Packet received from:
('192.168.1.10', 0)
b'E\x00\x00 <...'
```

RESULT: Thus, packet sniffing was successfully implemented using raw sockets in python, and network packets were captured and displayed.

EX NO: 9 Date: 13/08/2024

Design a Simple Topology and configure with one Router, TWO Switches and Pcs using CISCO Packet Tracer.

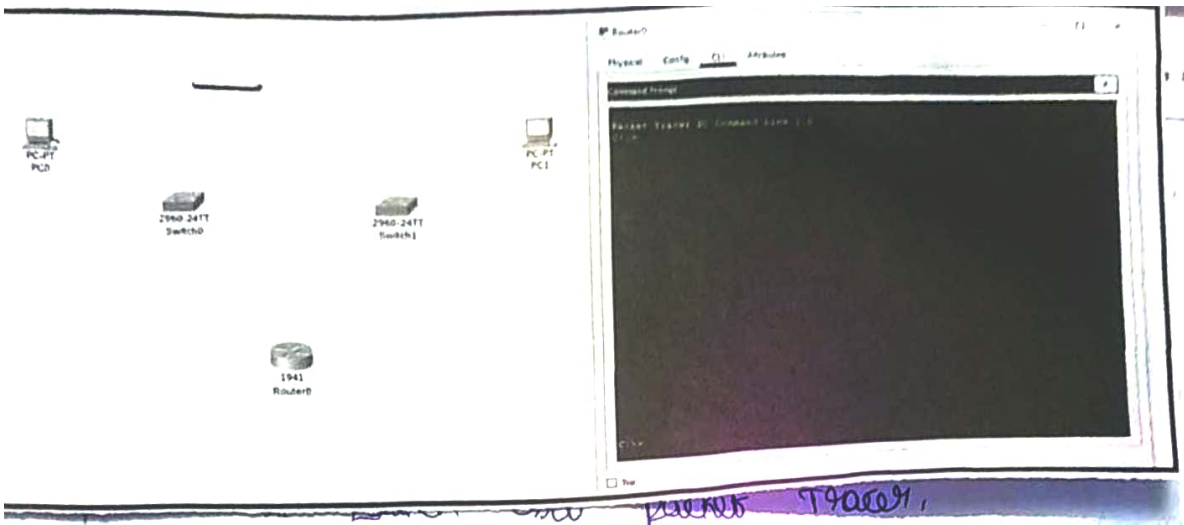
AIM: To design and configure a simple network topology with one router, two switches, and four Pcs in CISCO Packet Tracer and enable Communication between Pc's.

Definitions:

* **Router:** Forwards data packets between different networks.
* **Switch:** connects devices within a network and forwards data using MAC addresses.
* **Pc:** End device used to send and receive data.
* **Straight - through cable:** Used to Connect different devices such as Pc - switch and switch - Router.
* **Console Cable:** Used for router / switch configuration.

Equipment needed:

1. Cisco packet tracer.
2. 1 Router (1941)
3. 2 Switches (2960)



packet Tracer.

Step 2: Add devices.

* Add 1 Router (1941)
* Add 2 switches (2960)
* Add 4 PCs with 2PCs connected to each switch.

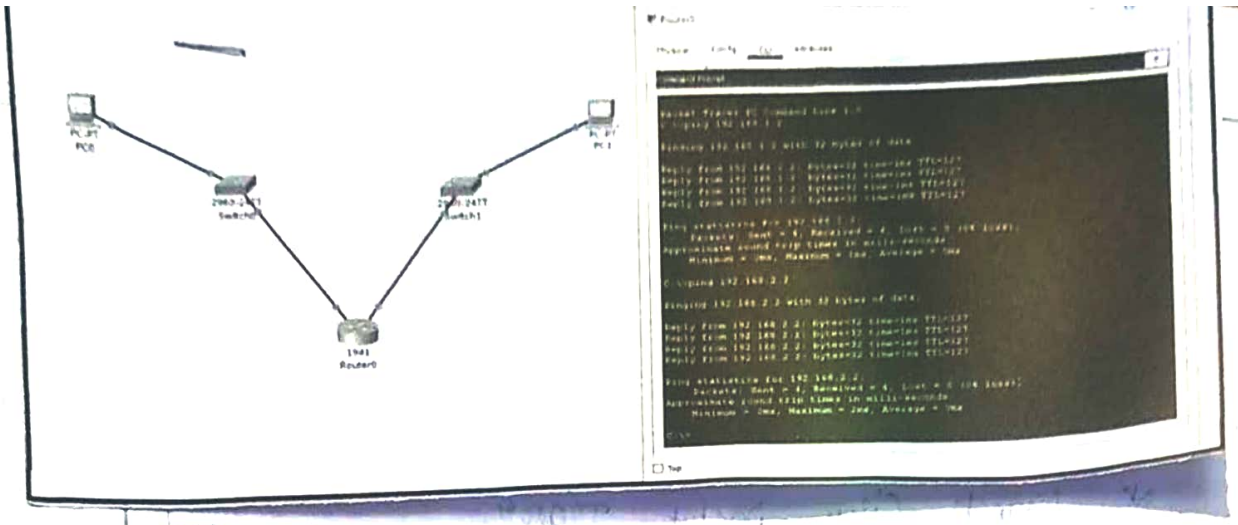
Step 3: Connect Devices.

Use Copper straigh-Through cables.

* Router Gigabit Ethernet 0/0 -> Switch 0 Fast Ethernet.
* Router Gigabit Ethernet 0/1 -> Switch 1 Fast Ethernet.
* PC0 -> switch 0 Fast Ethernet 0/2.
* PC1 -> Switch 0 Fast Ethernet 0/3.
* PC2 -> Switch 1 Fast Ethernet 0/2.
* PC3 -> Switch 1 Fast Ethernet 0/3.

Step 4: Configure Router Interfaces.

```
enable
configure terminal.
interface gigabit Ethernet 0/0
ip address 192.168.1.1 255.255.255.0
no shutdown
exit.
```



write memory.

Step 5: configure PC Ip Address.

Device	Ip address	Submit mask	Default Gateway
PC0	192.168.1.10	255.255.255.0	192.168.1.1
PC1	192.168.1.11	255.255.255.0	192.168.1.1
PC2	192.168.2.10	255.255.255.0	192.168.2.1
PC3	192.168.2.11	255.255.255.0	192.168.2.1

Step 6: Test Connectivity.

* Save the packet Tracer file.
* Open PC0 -> Command prompt.
* Enter
  Ping 192.168.2.10.
* Successful replies confirms proper network Configuration

Result: Thus designing a simple topology and Configure with one router, two switches, and PCs using CISCO packet Tracer is successfully verified.

EXP NO: 10 Date: 13/08/2026.

CUSTOMIZE SWITCH WITH NETWORK MODULES USING CISCO PACKET TRACER.

AIM: To know how to add a network module to a cisco switch, connect multiple PC's and a server, assign Ip addresses, and test connectivity in Cisco packet Tracer.

Description: This Experiment is about building a simple Computer network using a switch with extra module ports. you will connect five pc's and one server, assign IP's and check if all devices can communicate; this helps you understand how modular switches are used to create flexible networks.

Definition: A network module is an additional component added to a switch to provide more ports or connection types HWIC-4ESW is used in this experiment.

Procedure:

1. open cisco packet Tracer and setup your work-space.
2. Drag and drop a 2960-24TT switch to centre.

3. Add five PC's and one Server - PT from the End Devices panel.

4. power off the switch:

   Click the switch, open the physical tab, and turn off the power button.

5. Install HWIC-4ESW module on the switch,

6. Power on the switch and let it boot,

7. Connect all devices using Cooper Straight - Through Cables,

* PC0 -> Switch fast Ethernet 0/1.
* PC1 -> Switch Fast Ethernet 0/2.
* PC2 -> Switch Module port 1.
* PC3 -> Switch Module port 2,
* PC4 -> Switch module port 3 - server() -> Switch Module port 4. Assign Ip addresses to each device.

i) PC0 : 192.168.1.10,

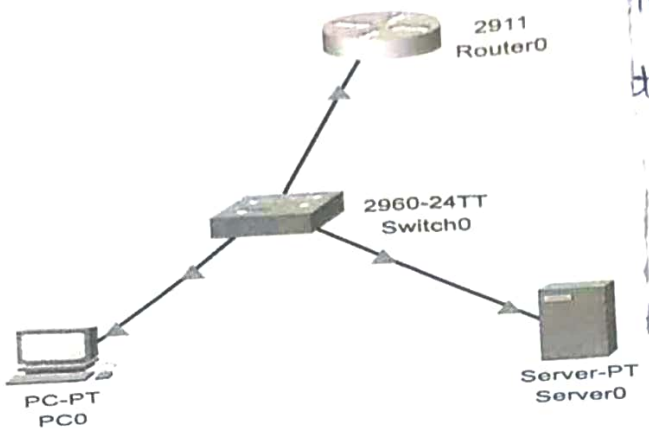
ii) PC1 : 192.168.1.11,

iii) PC2 : 192.168.1.12.

iv) PC3 : 192.168.1.13.

v) PC4 : 192.168.1.14,

vi) All Subnet masks : 255.255.255.0,



tivity (from any PC, open Command ther devices).

switch.
twork module.

* 5 PC's (PC0-PC4)
* 1 Server-PT (Server 0).

Cables:

* Copper straight-through cables IP configuration,
* Use Static IP addresses as listed above,

Output:

A working network setup in which all PC's and the server can successfully communicate confirming that the built-in and module ports are functioning.

Result: The switch was successfully customized with a network module, and all connected devices communicated successfully.

Date: 20.08.26.

Exp.no: 07.

Develop a simple Calculator using XML RPC.

AIM: To develop a simple calculator using XMLRPC in python, where a client can remotely call arithmetic functions provided by a server.

REQUIREMENTS:

* Python 3.X
* Python IDLE.
* XML - RPC Module.
* Two python programs: server.py and client.py.

PROCEDURE:

1. Open python IDLE.
2. Create a python file named server.py.
3. Write the XML - RPC server program containing the calculator functions.
4. Save and run server.py.
5. The message 'calculator server is running' is displayed.
6. Keep server running.
7. Open separate python IDLE window & create another file named client.py.

8. Run client.py.
9. Enter first and second numbers when prompted.
10. The server performs addition, subtraction, multiplication and division.
11. The results are returned and displayed.

Server program:

```
from xmlrpc.server import SimpleXMLRPCServer
def add(a,b):
    return a+b.
def subtract(a,b):
    return a-b
def multiply(a,b):
    return a*b
def divide(a,b):
    return a/b.
    if b==0:
        return "Cannot divide".
server = SimpleXMLRPCServer(("127.0.0.1", 8000))
server.register_function(add, "add"),
server.register_function(subtract, "subtract")
server.register_function(multiply, "multiply")
server.register_function(divide, "divide")
print("Calculator server is running...")
server.serve_forever()
```

Client program.

```
import xmlrpc.client
server = xmlrpc.client.ServerProxy("http://127.0.0.1:8000")
a = float(input("Enter first number: "))
b = float(input("Enter second number: "))
print("Addition = ", server.add(a,b))
print("Subtraction = ", server.subtraction(a,b))
print("Multiplication = ", server.multiply(a,b))
print("Division = ", server.divide(a,b))
```

Observation:

The XML-RPC server was successfully started and calculator functions were register. The client successfully connected to the server and remotely called the arithmetic functions. The calculated results were received from the server and displayed on the cl

```
from xmlrpc.server import SimpleXMLRPCServer
from xmlrpc.client import ServerProxy
import threading



def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def multiply(a, b):
    return a * b

def divide(a, b):
    if b == 0:
        return "Cannot divide by zero"
    return a / b
```

```
===== XML-RPC CALCULATOR =====
Enter first number: 2
Enter second number: 5

Choose operation:
1. Addition
2. Subtraction
3. Multiplication
4. Division
Enter your choice: 4

Result: 0.4
```

Result: Thus, a simple calculator was successfully developed and executed using XML-RPC in python. Th client-server communication and remote execution of arithmetic operations were successfully demonstrated.

Practical-6

AIM: Write a program to implement error detection and correction using HAMMING code concept. Make a test run to input data stream and verify error correction feature.

Error Correction at Data Link Layer:
Hamming code is a set of error-correction codes that can be used to detect and correct the errors that can occur when the data is transmitted from the sender to the receiver. It is a technique developed by R.W. Hamming for error correction.

Create sender program with below features.

1. Input to sender file should be a text of any length. Program should convert the text to binary.
2. Apply hamming code concept on the binary data and add redundant bits to it.
3. Save this output in a file called channel.

Create a receiver program with below features

1. Receiver program should read the input from Channel file.
2. Apply hamming code on the binary data to check for errors.
3. If there is an error, display the position of the error.
4. Else remove the redundant bits and convert the binary data to ascii and display the output.

Student observation:-

Write the code here:

Input:-

Output:

Practical-8

AIM: - To Discover Live Hosts Using Nmap Scans (ARP, ICMP, TCP/UDP) on the TryHackMe Platform Room Link :https://tryhackme.com/room/nmap01

Introduction

When targeting a network, we need an efficient tool to handle repetitive tasks. This tool should help us find out which systems are active and what services are running on those systems. The tool that we will rely on is Nmap. The first question about finding live computers is answered in this room. This room is the first in a series of four rooms dedicated to Nmap. The second question about discovering running services is answered in the next Nmap rooms that focus on port-scanning.

This room is the first of four in this Nmap series. These four rooms are also part of the Network Security module.

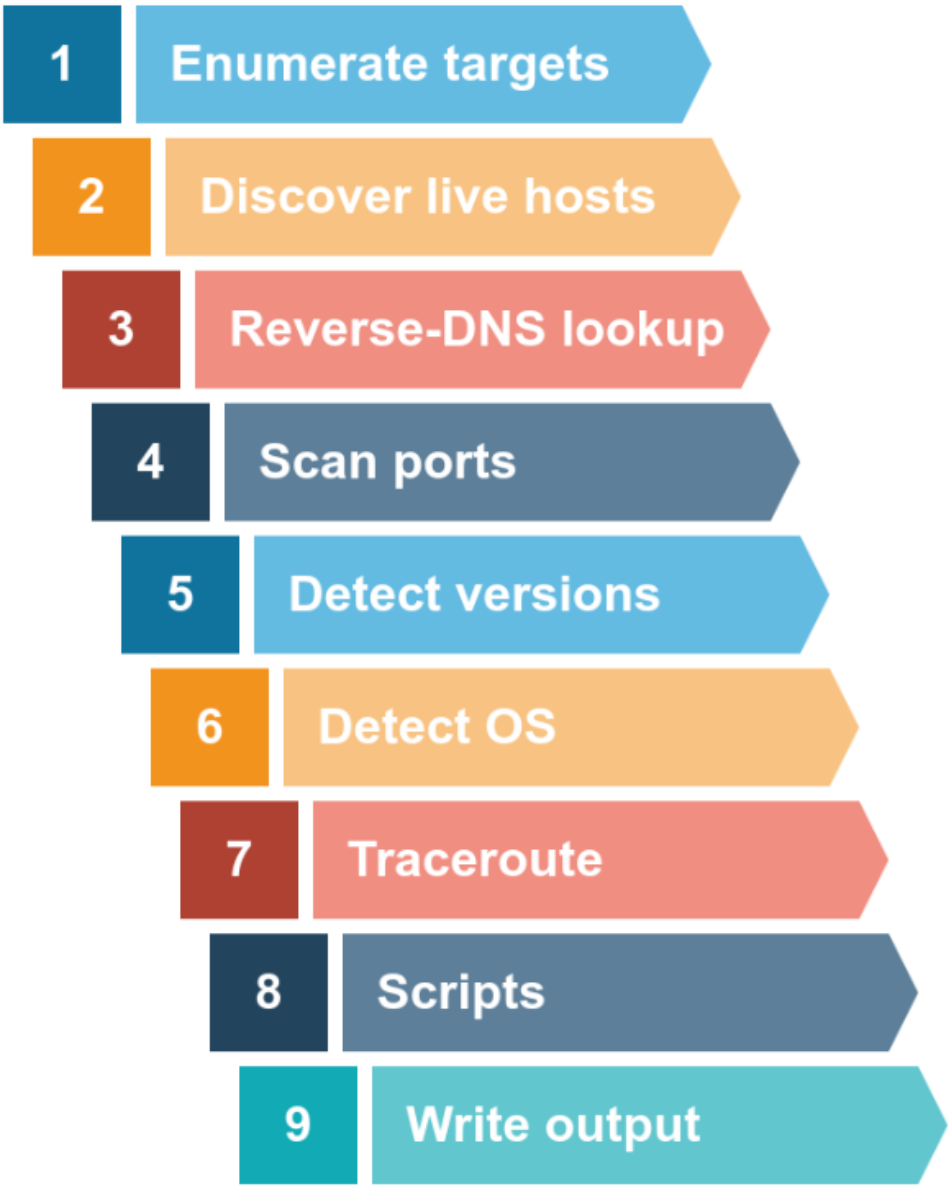
- Nmap Live Host Discovery
- Nmap Basic Port Scans
- Nmap Advanced Port Scans
- Nmap Post Port Scans

This room explains the steps that Nmap carries out to discover the systems that are online before port-scanning. This stage is crucial because trying to port-scan offline systems will only waste time and create unnecessary noise on the network.

We present the different approaches that Nmap uses to discover live hosts. In particular, we cover:
ARP scan: This scan uses ARP requests to discover live hosts
ICMP scan: This scan uses ICMP requests to identify live hosts
TCP/UDP ping scan: This scan sends packets to TCP ports and UDP ports to determine live hosts.

We also introduce two scanners, arp-scan and masscan, and explain how they overlap with part of Nmap's host discovery.

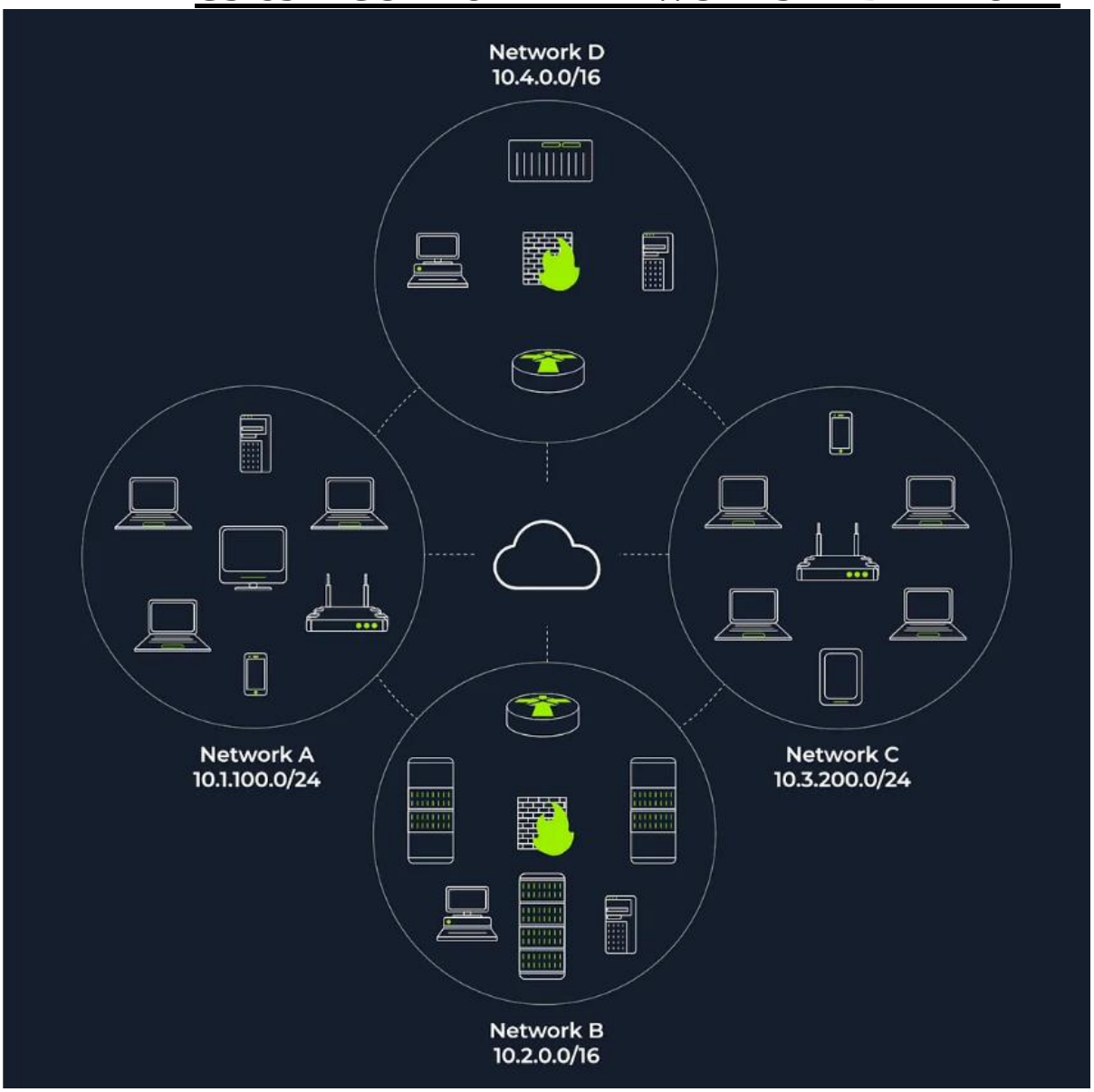
As already mentioned, starting with this room, we will use Nmap to discover systems and services actively. Nmap was created by Gordon Lyon (Fyodor), a network security expert and open source programmer. It was released in 1997. Nmap, short for Network Mapper, is free, open-source software released under GPL license. Nmap is an industry-standard tool for mapping networks, identifying live hosts, and discovering running services. Nmap's scripting engine can further extend its functionality, from fingerprinting services to exploiting vulnerabilities. A Nmap scan usually goes through the steps shown in the figure below, although many are optional and depend on the command-line arguments you provide.



Subnetworks

A network segment is a group of computers linked through a shared medium, like an Ethernet switch or WiFi access point. In IP networks, a subnetwork typically consists of one or more network segments connected together and configured to use the same router. A network segment is a physical connection, while a subnetwork is a logical connection.

In the provided network diagram, there are four network segments or subnetworks. Your system would usually connect to one of these segments/subnetworks. Each subnet has its own IP address range and is connected to a larger network through a router. Depending on the network, there might be a firewall enforcing security policies.

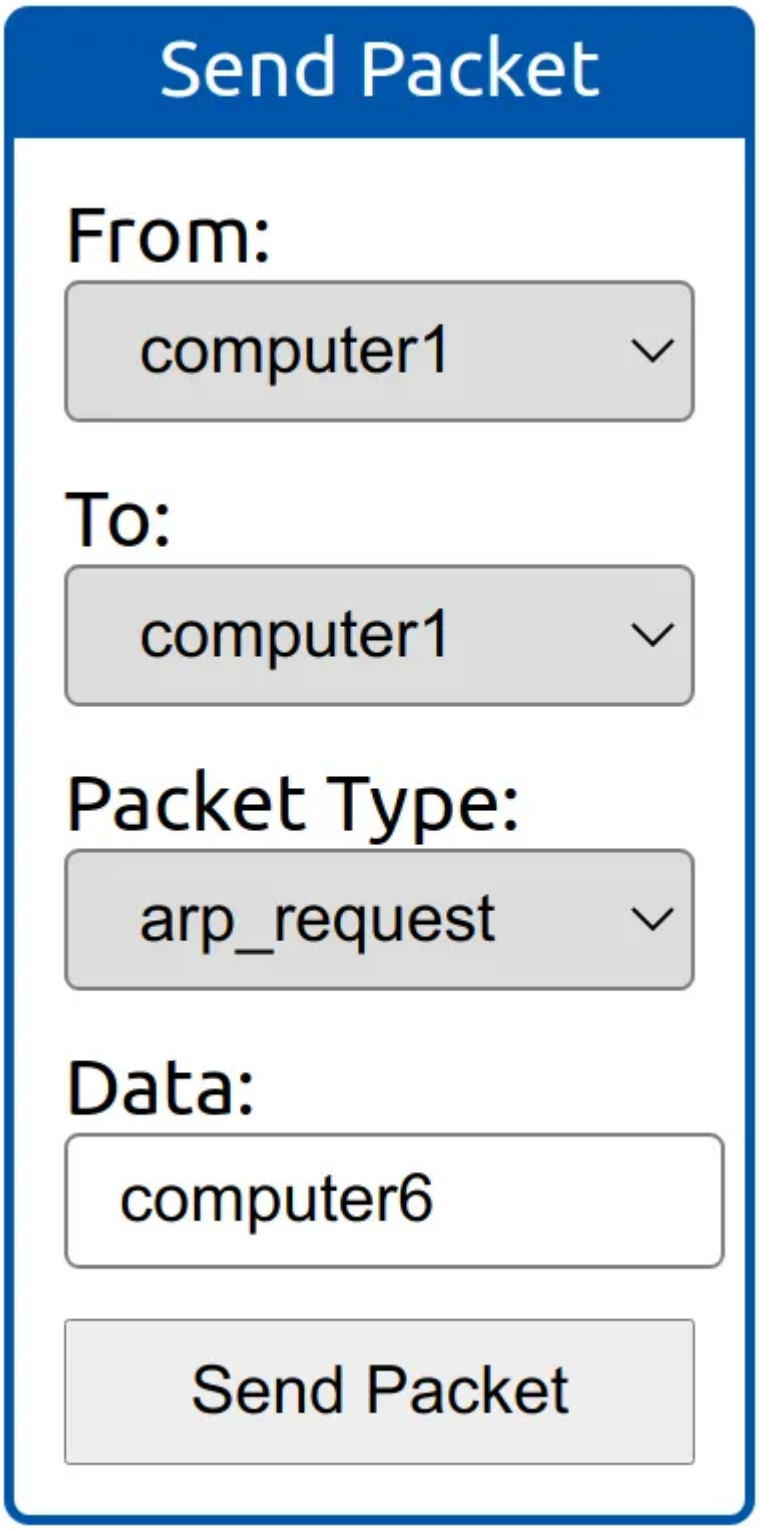


The figure displays two types of subnets:

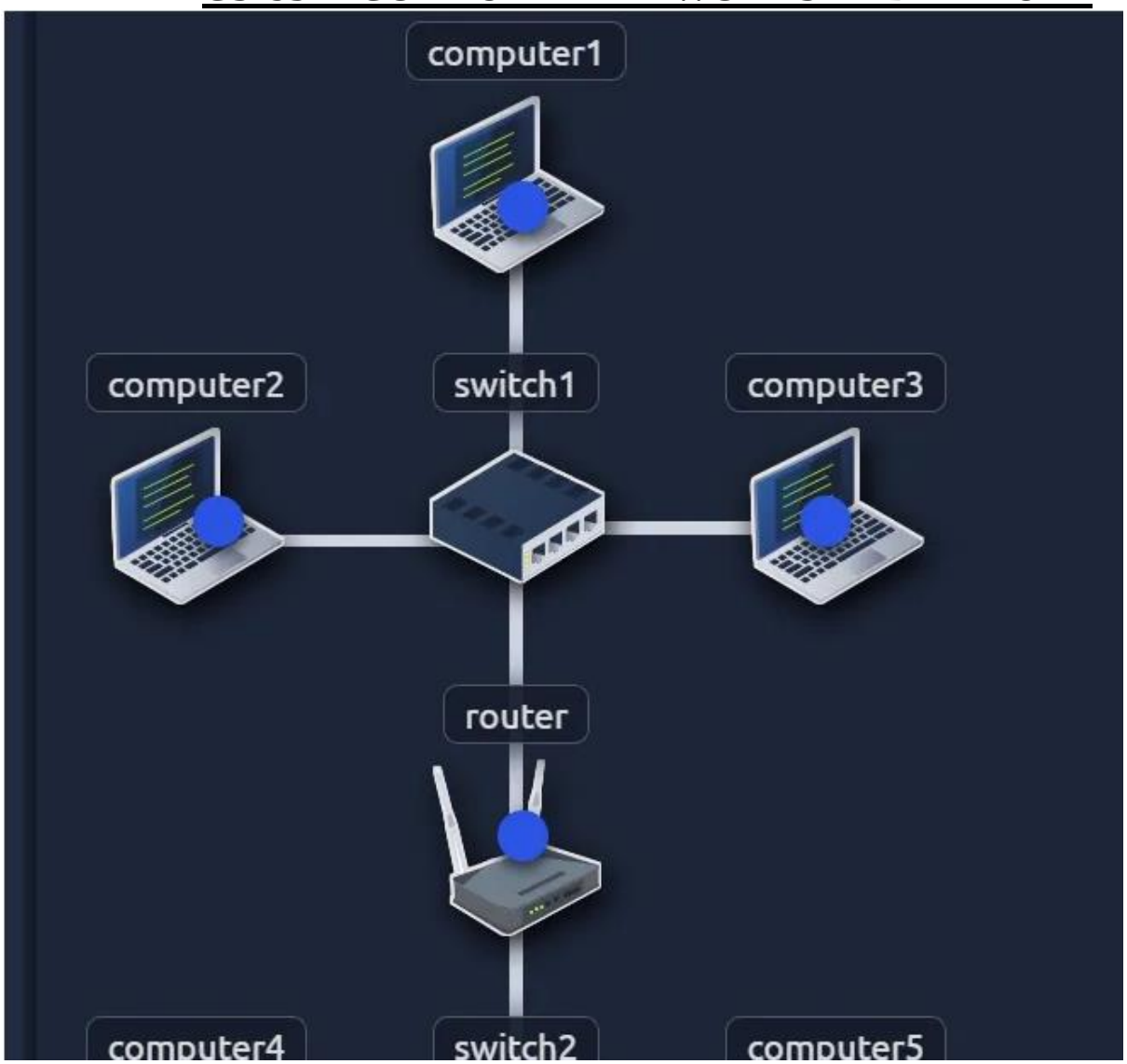
/16 Subnets: These have a subnet mask of 255.255.0.0 and can accommodate approximately 65 thousand hosts.

/24 Subnets: These feature a subnet mask of 255.255.255.0 and can support around 250 hosts.

In active reconnaissance, when attempting to gather information about a group of hosts or a subnet, if you're on the same subnet, your scanner relies on ARP (Address Resolution Protocol) queries to find live hosts. ARP queries seek to obtain the MAC address, enabling link-layer communication, which implies the host is online. However, ARP can only discover devices within the same subnet. If you're on a different subnet from the target, your scanner's packets will be routed through the default gateway, but ARP queries cannot cross subnet routers since ARP packets are tied to their specific subnet due to being a link-layer protocol.



- from computer1
- To computer1 (to indicate it is broadcast)
- Packet Type: “ARP Request”
- Data: computer6 (because we are asking for computer6 MAC address using ARP Request)

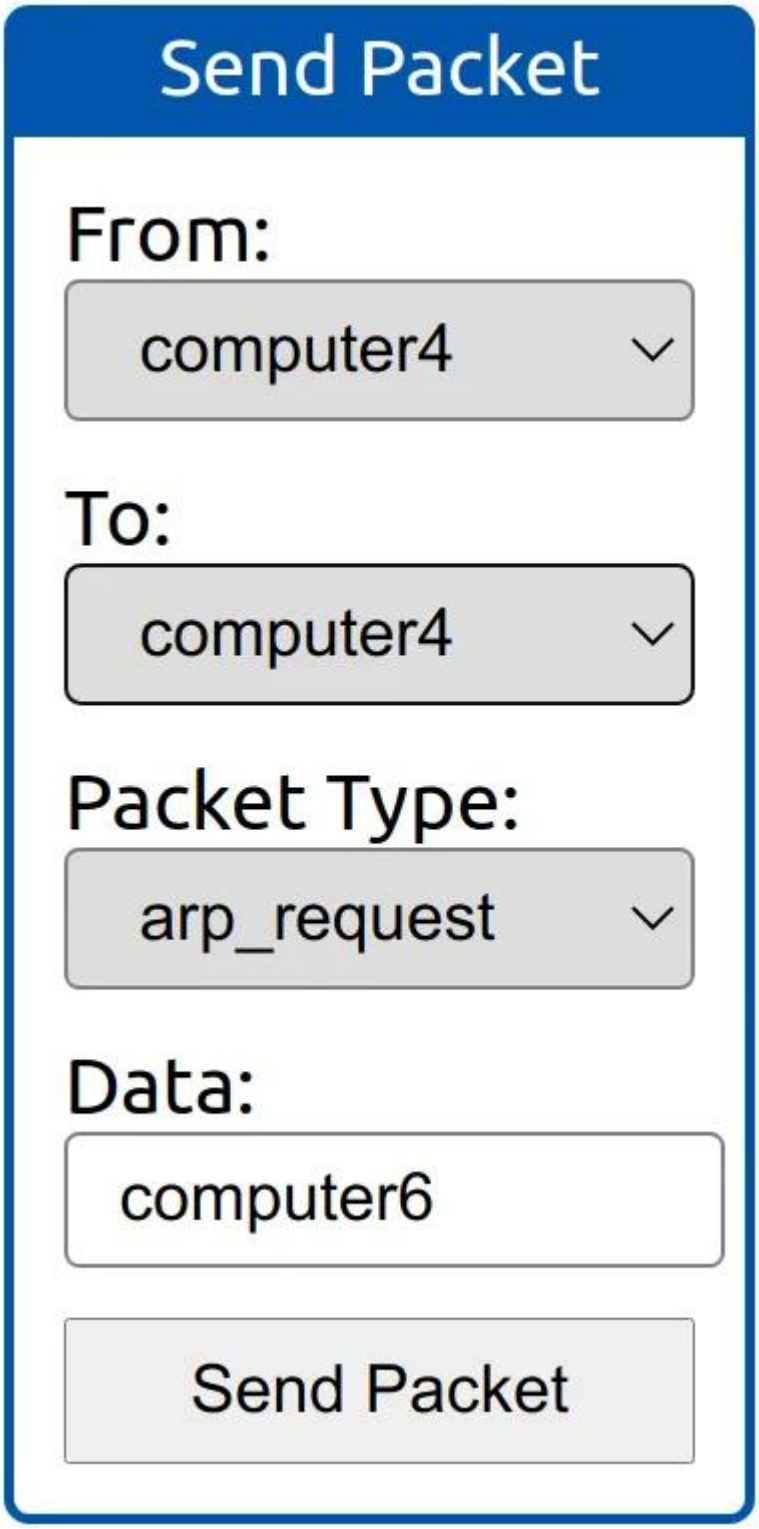


How many devices can see the ARP Request?

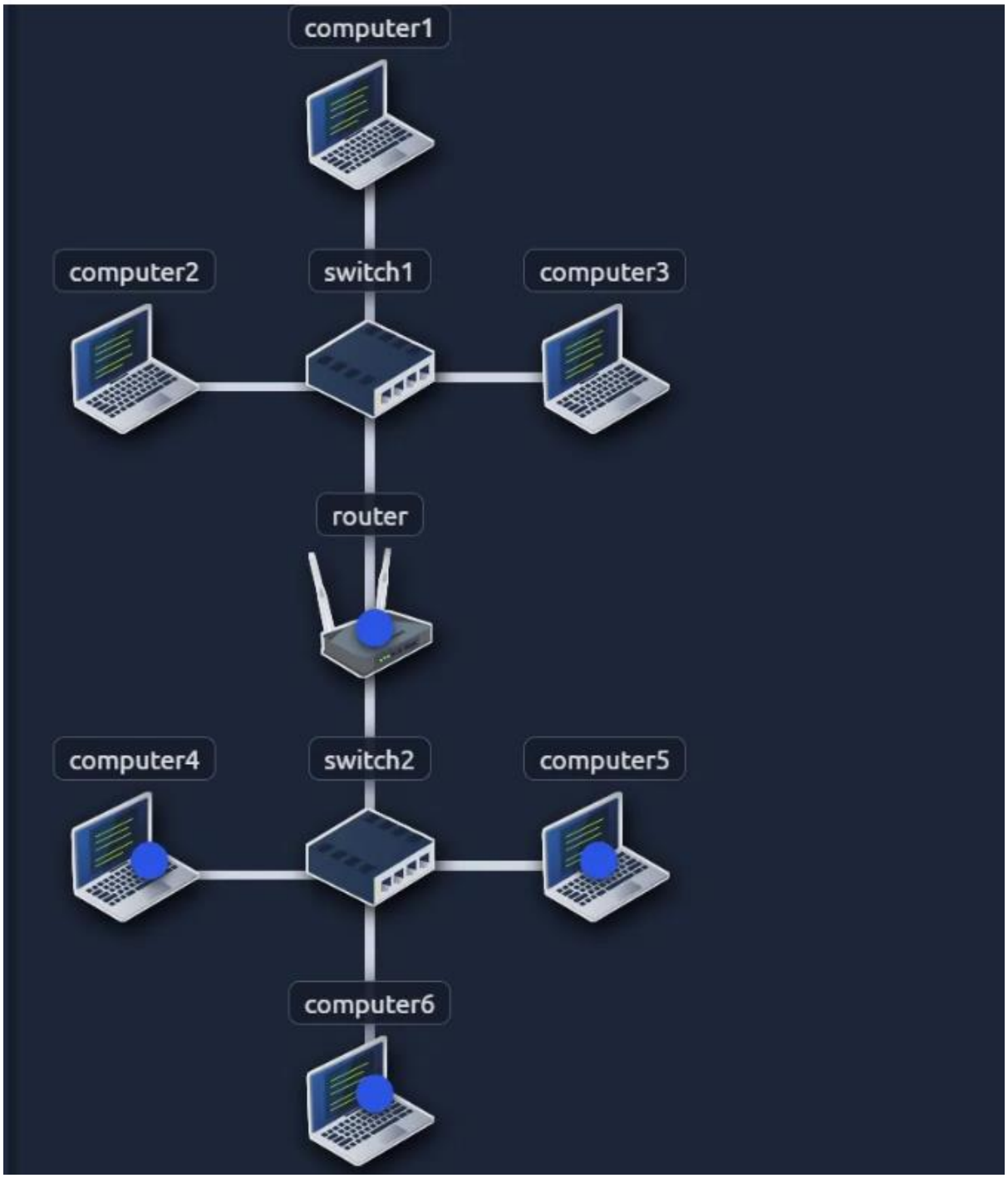
4

Did computer6 receive the ARP Request? (Y/N)

N



- From computer4
- To computer4 (to indicate it is broadcast)
- Packet Type: “ARP Request”
- Data: computer6 (because we are asking for computer6 MAC address using ARP Request)



How many devices can see the ARP Request?
4
Did computer6 reply to the ARP Request? (Y/N)
Y

Enumerating Targets

Before delving into the details of scanning techniques, it's essential to define the targets we want to scan. Targets can be specified in three ways:

1. List: You can provide a list of specific IP addresses or hostnames, like "MACHINE_IP," "scanme.nmap.org," and "example.com," which would result in scanning 3 IP addresses.
2. Range: You can specify a range, such as "10.11.12.15–20," which will scan 6 IP addresses: 10.11.12.15, 10.11.12.16, and so on, up to 10.11.12.20.

3. Subnet: You can define a subnet like "MACHINE_IP/30," which will scan 4 IP addresses within that subnet.

Nmap allows you to input a list of targets from a file using "nmap -iL list_of_hosts.txt." You can also preview the list of hosts that Nmap intends to scan by using "nmap -sL TARGETS," which provides a detailed list without actually scanning them. However, Nmap will attempt reverse-DNS resolution to obtain host names, potentially revealing valuable information to the pentester. To prevent DNS resolution, you can add the "-n" flag.

What is the first IP address Nmap would scan if you provided 10.10.12.13/29 as your target?

=>/29= 8 address,

=> 10.10.12.[0–7][8–15]

=>10.10.12.8

How many IP addresses will Nmap scan if you provide the following range 10.10.0-255.101-125?

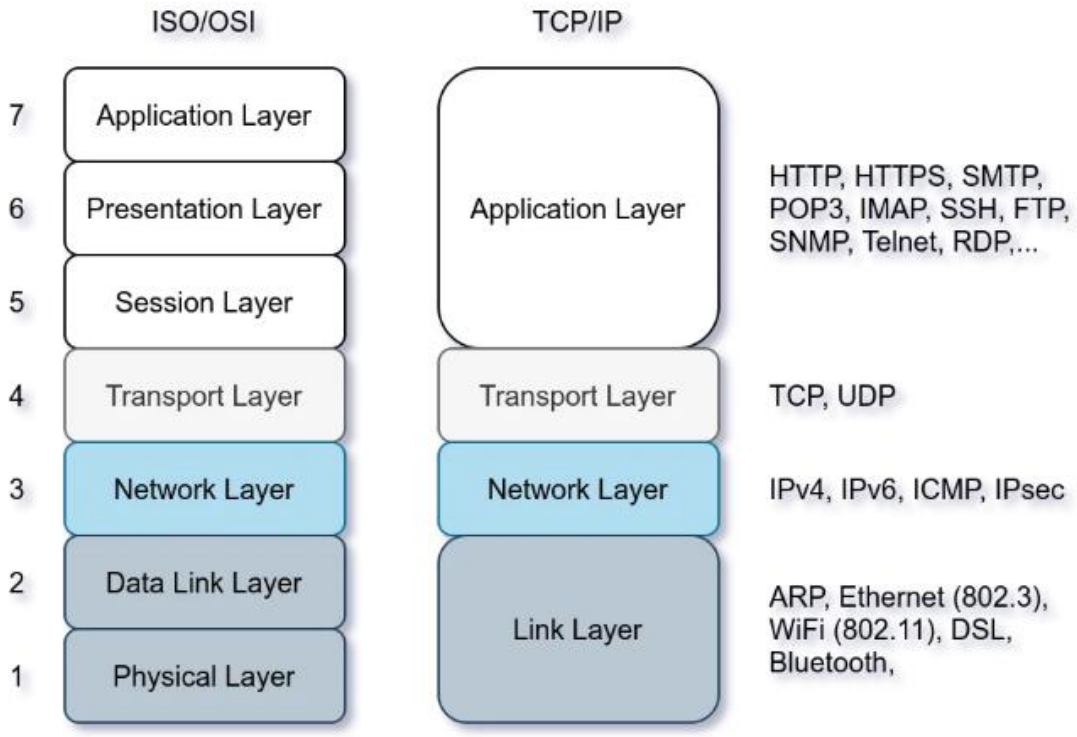
*=> 255*25*

=>6400

Discvoering Live hosts

TCP/IP layers:

- ARP from Link Layer
- ICMP from Network Layer
- TCP from Transport Layer
- UDP from Transport Layer



This passage discusses four network protocols: ARP, ICMP, TCP, and UDP, and their roles in network scanning. ARP is used to request a computer's MAC address with a specific IP address. ICMP has various types, including ping (Type 8 and Type 0). When pinging a system on the same subnet, an ARP query should be sent before ICMP Echo. Additionally, network scanners can use specially-crafted packets to common TCP or UDP ports for efficient target response checking, especially when ICMP Echo is blocked.

Send a packet with the following:

- From computer1
- To computer3
- Packet Type: “Ping Request”

What is the type of packet that computer1 sent before the ping?
ARP Request
What is the type of packet that computer1 received before being able to send the ping?
ARP Response
How many computers responded to the ping request?
1
Send a packet with the following:

- From computer2
- To computer5
- Packet Type: “Ping Request”

What is the name of the first device that responded to the first ARP Request?
router
What is the name of the first device that responded to the second ARP Request?
computer5
Send another Ping Request. Did it require new ARP Requests? (Y/N)
N
NMAP Host Discovery Using ARP

How can we determine which hosts are operational? It’s crucial to prevent unnecessary port scanning on hosts that are offline or not in use. There are several methods to identify active hosts. When no specific host discovery options are specified, Nmap employs the following strategies to find live hosts:

When a privileged user tries to scan targets on a local network (Ethernet), Nmap uses ARP requests. A privileged user is root or a user who belongs to sudoers and can run sudo.

When a privileged user tries to scan targets outside the local network, Nmap uses ICMP echo requests, TCP ACK (Acknowledge) to port 80, TCP SYN (Synchronize) to port 443, and ICMP timestamp request.

When an unprivileged user tries to scan targets outside the local network, Nmap resorts to a TCP 3-way handshake by sending SYN packets to ports 80 and 443.

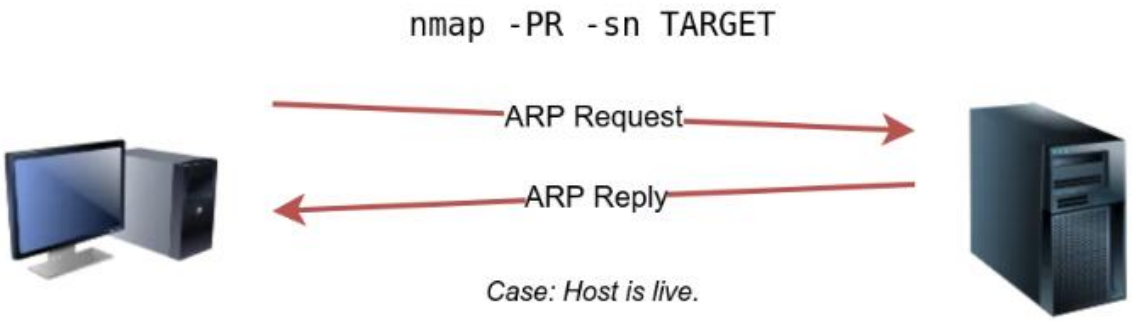
Nmap typically uses a ping scan to find live hosts and then proceeds to scan those live hosts. However, you can use the “nmap -sn TARGETS” command to discover online hosts without conducting port scans. ARP scan is one such method, but it only works when you are on the same subnet as the target systems because it relies on MAC addresses for communication. ARP queries are sent to obtain MAC addresses, and hosts that respond to these queries are considered up. You may see many ARP queries during a local network scan with Nmap. To perform only an ARP scan without port scanning, you can use “nmap -PR -sn TARGETS,” where “-PR” specifies an ARP scan. This allows you to discover live systems on the same subnet as your target machine without conducting any port scans.

```
Pentester Terminal

pentester@TryHackMe$ sudo nmap -PR -sn 10.10.210.6/24

Starting Nmap 7.60 ( https://nmap.org ) at 2021-09-02 07:12 BST
Nmap scan report for ip-10-10-210-75.eu-west-1.compute.internal (10.10.210.75)
Host is up (0.00013s latency).
MAC Address: 02:83:75:3A:F2:89 (Unknown)
Nmap scan report for ip-10-10-210-100.eu-west-1.compute.internal (10.10.210.100)
Host is up (-0.100s latency).
MAC Address: 02:63:D0:1B:2D:CD (Unknown)
Nmap scan report for ip-10-10-210-165.eu-west-1.compute.internal (10.10.210.165)
Host is up (0.00025s latency).
MAC Address: 02:59:79:4F:17:B7 (Unknown)
Nmap scan report for ip-10-10-210-6.eu-west-1.compute.internal (10.10.210.6)
Host is up.
Nmap done: 256 IP addresses (4 hosts up) scanned in 3.12 seconds
```

In this scenario, the AttackBox had the IP address 10.10.210.6 and employed ARP requests to identify active hosts within the same subnet. Nmap sends ARP requests to all the target machines, and those that are online will respond with an ARP reply. The ARP scan operates as depicted in the accompanying figure.



- From computer1
- To computer1 (to indicate it is broadcast)
- Packet Type: “ARP Request”

Data: try all the possible eight devices (other than computer1) in the network: computer2, computer3, computer4, computer5, computer6, switch1, switch2, and router.

How many devices are you able to discover using ARP requests?

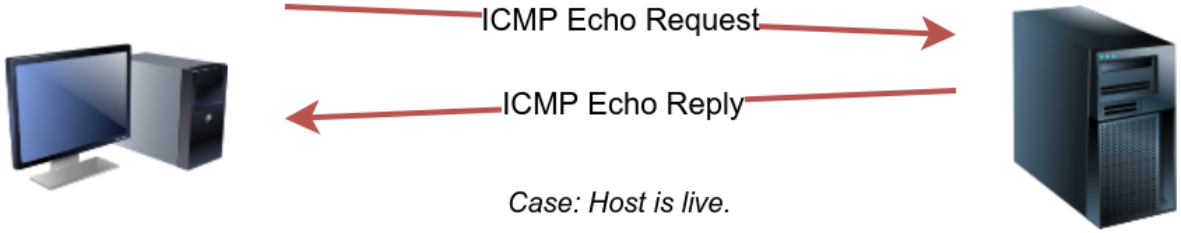
3

NMAP Host Discovery Using ICMP

A simple method to identify live hosts on a target network is by pinging each IP address and checking for responses (ICMP Type 8/Echo requests and Type 0/Echo replies). However, this approach is not always reliable because some firewalls block ICMP echo requests, and newer versions of Windows have default settings that do so as well. If the target is on the same subnet, an ARP query will precede the ICMP request. To perform host discovery using ICMP echo requests, you can use the option “-PE” and include “-sn” if you don’t want to conduct a subsequent port scan.

As shown in the following figure, an ICMP echo scan works by sending an ICMP echo request and expects the target to reply with an ICMP echo reply if it is online.

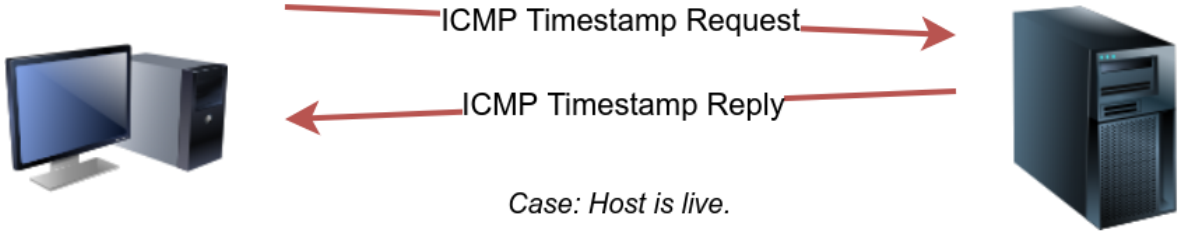
```
nmap -PE -sn TARGET
```



Case: Host is live.

Nmap uses timestamp request (ICMP Type 13) and checks whether it will get a Timestamp reply (ICMP Type 14). Adding the -PP option tells Nmap to use ICMP timestamp requests. As shown in the figure below, you expect live hosts to reply.

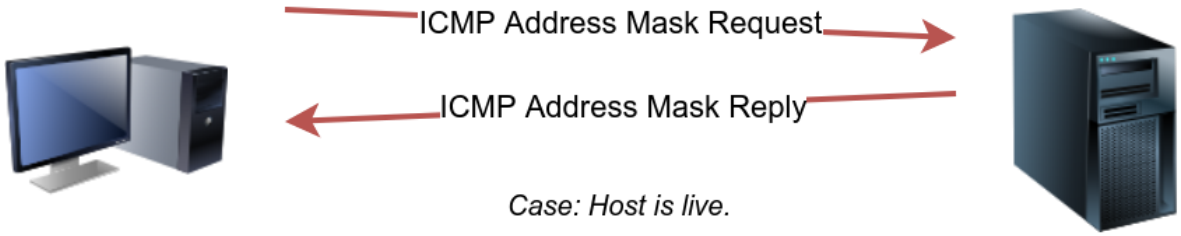
```
nmap -PP -sn TARGET
```



Case: Host is live.

Similarly, Nmap uses address mask queries (ICMP Type 17) and checks whether it gets an address mask reply (ICMP Type 18). This scan can be enabled with the option -PM. As shown in the figure below, live hosts are expected to reply to ICMP address mask requests.

```
nmap -PM -sn TARGET
```



Case: Host is live.

Answer the questions below

What is the option required to tell Nmap to use ICMP Timestamp to discover live hosts?

-PP

What is the option required to tell Nmap to use ICMP Address Mask to discover live hosts?

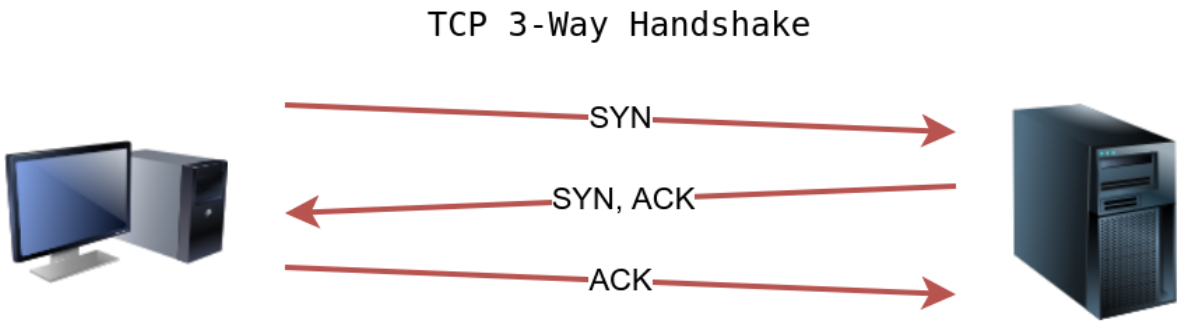
-PM

What is the option required to tell Nmap to use ICMP Echo to discover live hosts?

-PE

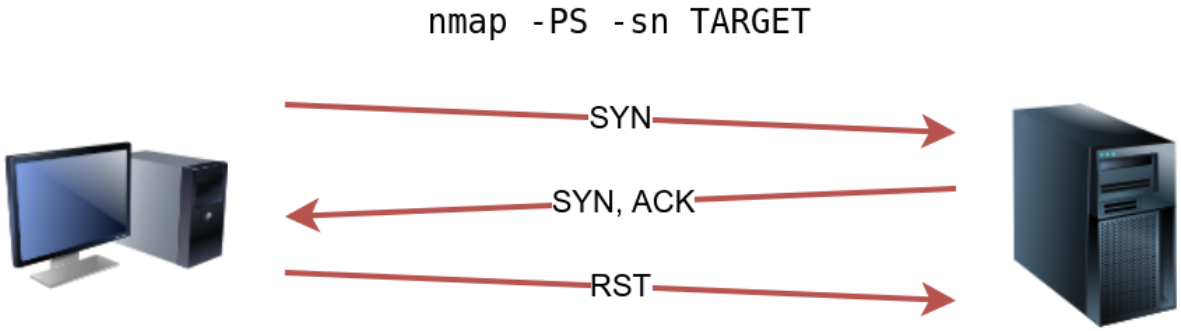
NMAP Host Discovery Using TCP AND UDP

TCP SYN Ping



Case: TCP port is open.

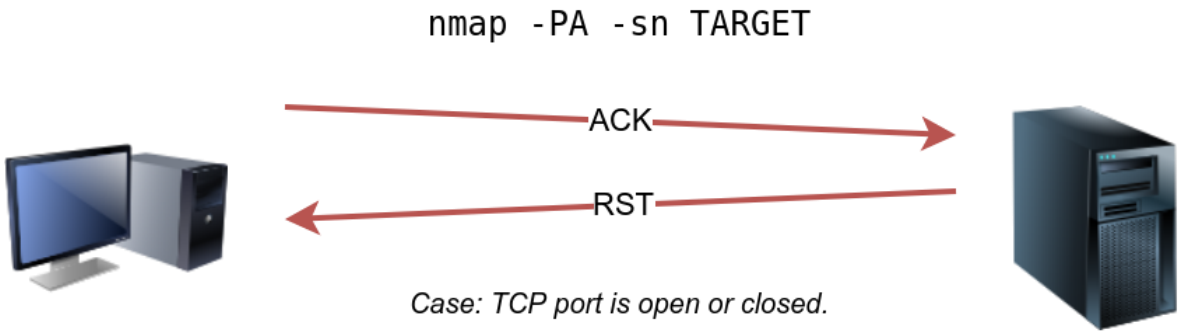
To determine if a host is up, you can send a packet with the SYN (Synchronize) flag set to a default TCP port, usually 80, and wait for a response. An open port will reply with SYN/ACK, while a closed port will result in an RST. In this method, the specific state of the port is not crucial; it's about checking for any response to confirm the host's status. You can enable Nmap to use TCP SYN ping with the option "-PS" followed by the port number, range, or list. For example, "-PS21" targets port 21, while "-PS21–25" targets ports 21 to 25. Privileged users can send TCP SYN packets without completing the 3-way handshake, unlike unprivileged users who must complete it if the port is open.



Case: TCP port is open.

TCP ACK Ping

To utilize ACK ping in Nmap, which sends a packet with the ACK flag set, you need to run Nmap as a privileged user. If you attempt this as an unprivileged user, Nmap will perform a 3-way handshake by default.



Case: TCP port is open or closed.

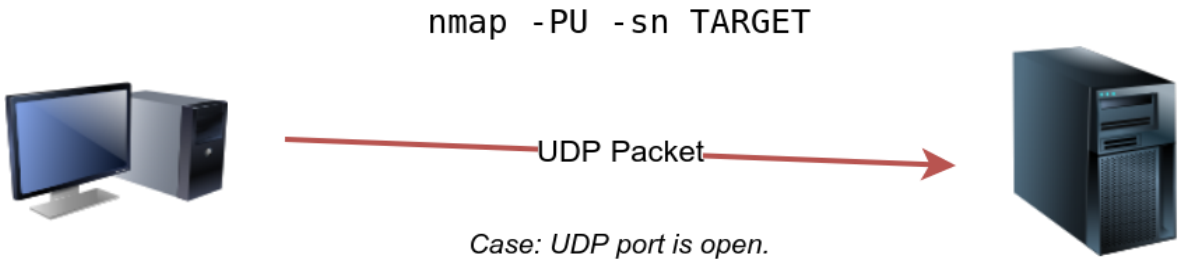
By default, Nmap uses port 80, and you can specify the port(s) using the "-PA" option, followed by a port number, range, list, or a combination thereof. For instance, you can use "-PA21," "-PA21–25," or "-PA80,443,8080." If no port is specified, Nmap will use port 80.

The expected response for an ACK ping is a TCP packet with the RST flag set because the ACK packet is not part of an established connection. This response helps determine if the target host is up.

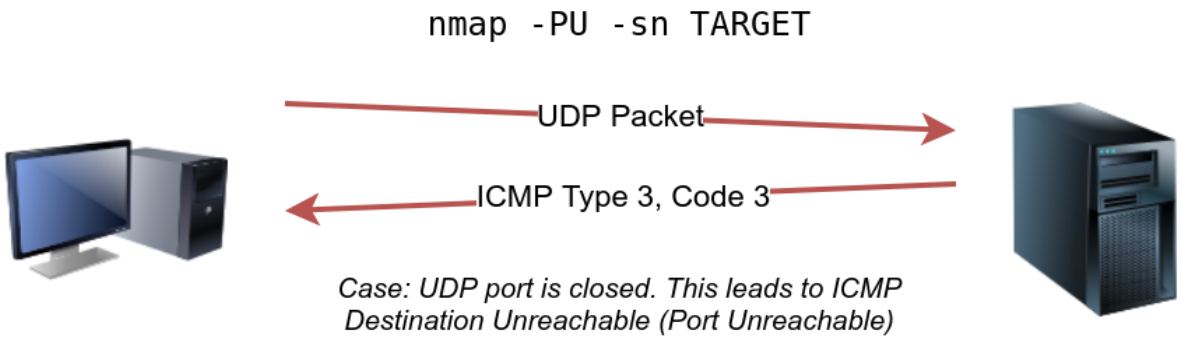
UDP Ping

You can also employ UDP to check if the host is online. Unlike TCP SYN ping, sending a UDP packet to an open port typically doesn't elicit a response. However, when sending a UDP packet to a closed UDP port, you anticipate receiving an ICMP "port unreachable" packet, which indicates the target system is active and reachable.

In summary, while sending UDP packets to open UDP ports may not trigger a response, sending them to closed UDP ports can indirectly indicate that the target is online, as it may generate a "port unreachable" ICMP message.



Case: UDP port is open.



Case: UDP port is closed. This leads to ICMP Destination Unreachable (Port Unreachable)

Answer the questions below

Which TCP ping scan does not require a privileged account?

TCP SYN Ping

Which TCP ping scan requires a privileged account?

TCP ACK Ping

What option do you need to add to Nmap to run a TCP SYN ping scan on the telnet port?

-PS23

Using Reverse-DNS Lookup

Nmap's standard operation is to perform reverse-DNS lookups for online hosts, which can provide valuable information through hostnames. If you prefer not to conduct these DNS queries, you can use the "-n" option to bypass this process.

By default, Nmap performs DNS queries for online hosts, but you can use the "-R" option to query the DNS server even for hosts that are offline. Additionally, if you wish to specify a particular DNS server, you can include the " — dns-servers DNS_SERVER" option.

We want Nmap to issue a reverse DNS lookup for all the possibles hosts on a subnet, hoping to get some insights from the names. What option should we add?

-R

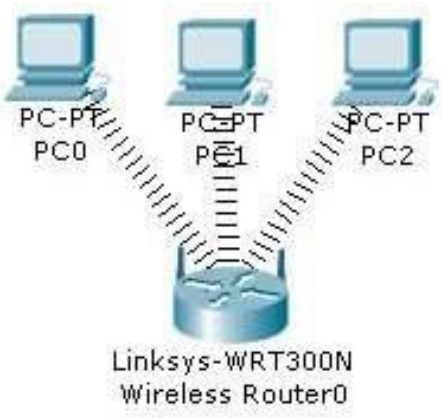
Summary

We have learned how ARP, ICMP, TCP, and UDP can detect live hosts by completing this room.

Practical-8

AIM:-b) Configuration of Wireless LAN using CISCO Packet Tracer.

Design a topology with three PCs connected from Linksys Wireless routers.



Perform following configuration:-

- Configure Static IP on PC and Wireless Router
- Set SSID to MotherNetwork
- Set IP address of router to 192.168.0.1, PC0 to 192.168.0.2, PC1 to 192.168.0.3 and PC2 to 192.168.0.4.
- Secure your network by configuring WAP key on Router
- Connect PC by using WAP key

To complete these tasks follow these step by step instructions:-

Step1:- Click on wireless router,

- Select Administration tab from top Menu, set username and password to admin and click on Save Setting.



- Next click on wireless tab and set default SSID to MotherNetwork.
- Now Select wireless security and change Security Mode to WEP



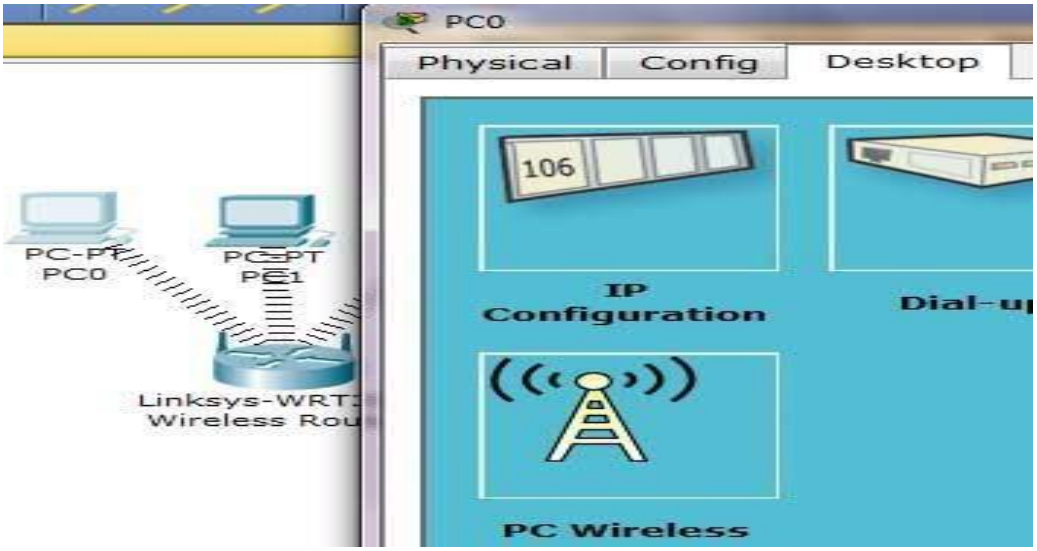
- Set Key1 to 0123456789



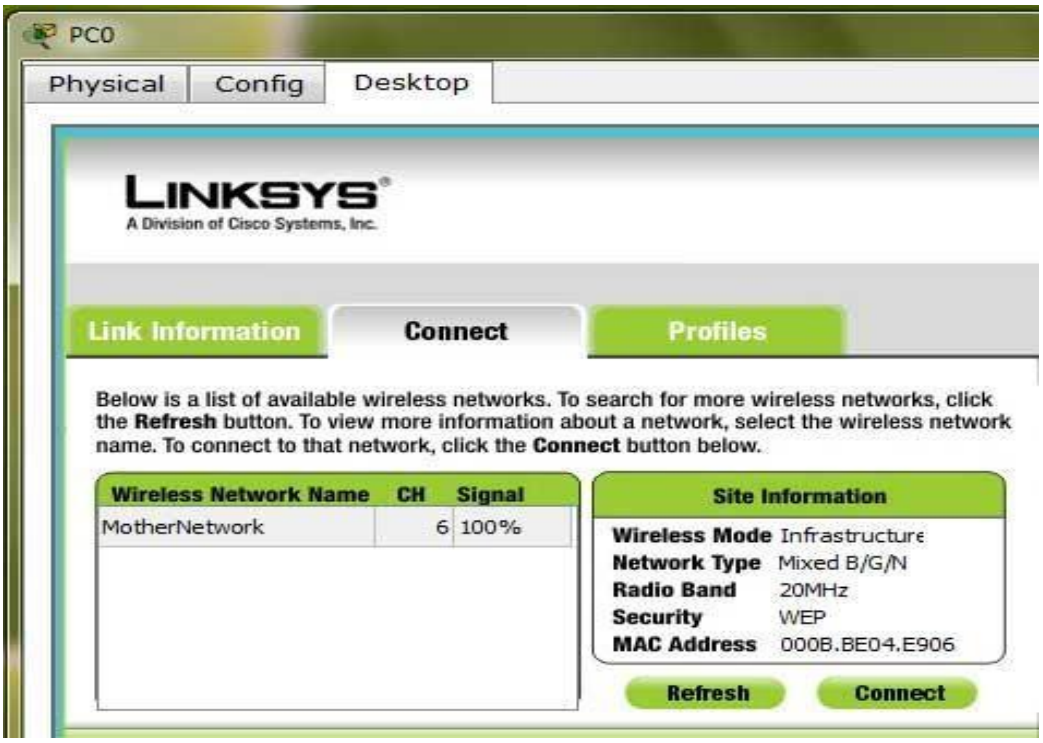
- Again go in the end of page and Click on Save Setting
- Now we have completed all given task on Wireless router. Now configure the static IP on all three PC's
- Double click on pc select Desktop tab click on IP configuration select Static IP and set IP as given below

PC	IP	Subnet Mask	Default Gateway
PC0	192.168.0.2	255.255.255.0	192.168.0.1
PC1	192.168.0.3	255.255.255.0	192.168.0.1
PC2	192.168.0.4	255.255.255.0	192.168.0.1

- Now it's time to connect PC's from Wireless router. To do so click PC select Desktop click on PC Wireless

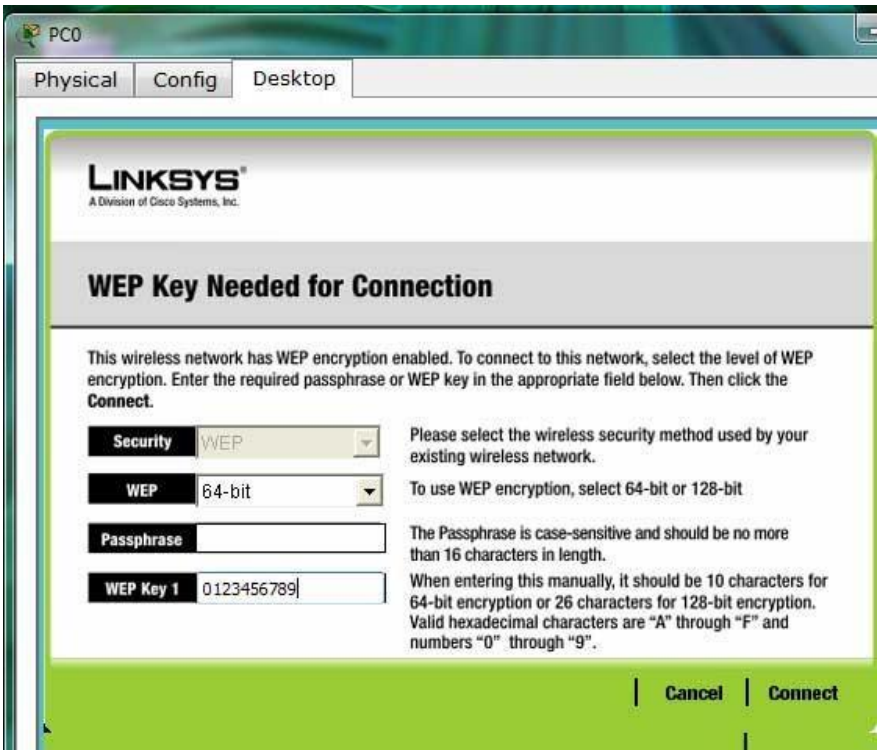


- Click on connect tab and click on Refresh button



As you can see in image that Wireless device is accessing MotherNetwork on CH 6 and signal strength is 100%. In left side you can see that WEP security is configured in network. Click on connect button to connect MotherNetwork

- It will ask for WAP key insert 0123456789 and click connect



It will connect you with wireless router.

As you can see in image below that system is connected. And PCI card is active.



- Repeat same process on PC1 and PC2.

Student observation:

c) **What is SSID of a wireless router?**
d) **What is a security key in wireless router?**
e) **Configure a simple Wireless LAN in your lab using a real access point and write down the configurations in your notebook.**

Practical 11

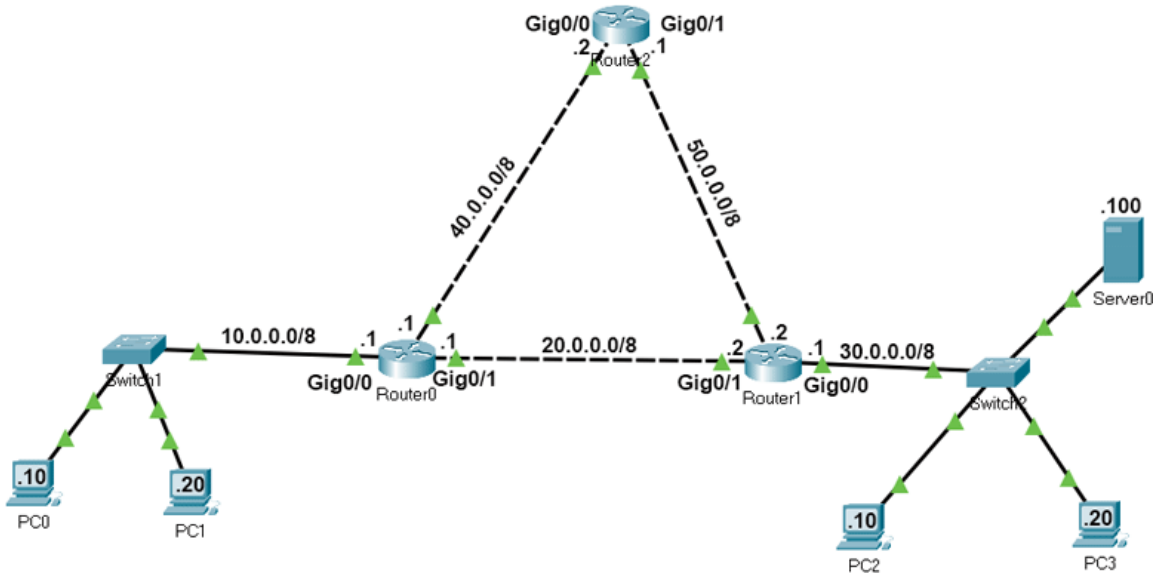
AIM:- a)Simulate Static Routing Configuration using CISCO Packet Tracer

Static routes are the routes you manually add to the router's routing table. The process of adding static routes to the routing table is known as static routing. Let's take a packet tracer example to understand how to use static routing to create and add a static route to the routing table.

Setting up a practice lab

Create a packet tracer lab as shown in the following image or download the following pre-created lab and load it on Packet Tracer.

Packet Tracer Lab with Initial IP Configuration



In this lab, each network has two routes to reach. We will configure one route as the main route and another route as the backup route. If the link bandwidth of all routes is the same, we use the route that has the least number of routers as the main route. If the link bandwidth and the number of routers are the same, we can use any route as the main route and another route as the backup route.

If we specify two routes for the same destination, the router automatically selects the best route for the destination and adds the route to the routing table. If you manually want to select a route that the router should add to the routing table, you have to set the AD value of the route lower than other routes. For example, if you use the following commands to create two static routes for network 30.0.0/8, the route will place the first route to the routing table.

```
#ip route 30.0.0.0 255.0.0.0 20.0.0.2 10
#ip route 30.0.0.0 255.0.0.0 40.0.0.2 20
```

If the first route fails, the router automatically adds the second route to the routing table.

Creating, adding, verifying static routes

Routers automatically learn their connected networks. We only need to add routes for the networks that are not available on the router's interfaces. For example, network 10.0.0.0/8, 20.0.0.0/8 and 40.0.0.0/8 are directly connected to Router0. Thus, we don't need to configure routes for these

networks. Network 30.0.0.0/8 and network 50.0.0.0/8 are not available on Router0. We have to create and add routes only for these networks.

The following table lists the connected networks of each router.

Router	Available networks on local interfaces	Networks available on other routers' interfaces
Router0	10.0.0.0/8, 20.0.0.0/8, 40.0.0.0/8	30.0.0.0/8, 50.0.0.0/8
Router1	20.0.0.0/8, 30.0.0.0/8, 50.0.0.0/8	10.0.0.0/8, 40.0.0.0/8
Router2	40.0.0.0/8, 50.0.0.0/8	10.0.0.0/8, 20.0.0.0/8, 30.0.0.0/8

Let's create static routes on each router for networks that are not available on the router.

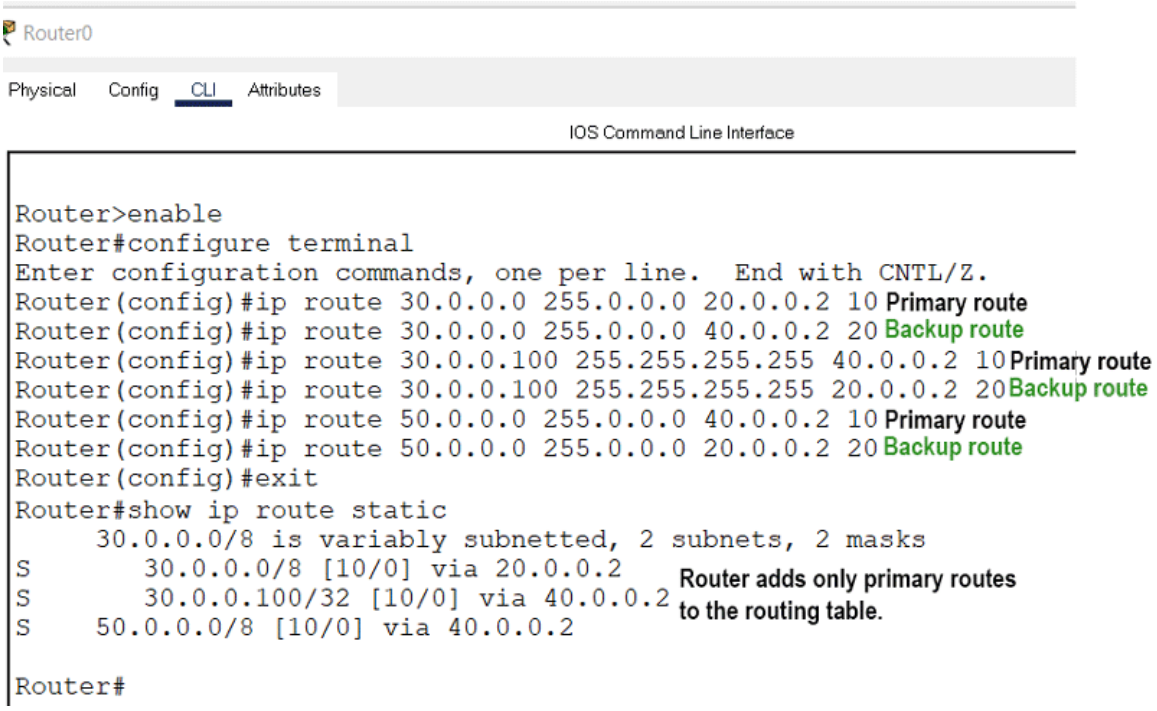
Router0 requirements

- Create two routes for network 30.0.0.0/8 and configure the first route (via -Router1) as the main route and the second route (via-Router2) as a backup route.
- Create two routes for the host 30.0.0.100/8 and configure the first route (via -Router2) as the main route and the second route (via-Router1) as a backup route.
- Create two routes for network 50.0.0.0/8 and configure the first route (via -Router2) as the main route and the second route (via-Router1) as a backup route.
- Verify the router adds only main routes to the routing table.

Router0 configuration

Access the CLI prompt of Router0 and run the following commands.

```
Router>enable
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#ip route 30.0.0.0 255.0.0.0 20.0.0.2 10
Router(config)#ip route 30.0.0.0 255.0.0.0 40.0.0.2 20
Router(config)#ip route 30.0.0.100 255.255.255.255 40.0.0.2 10
Router(config)#ip route 30.0.0.100 255.255.255.255 20.0.0.2 20
Router(config)#ip route 50.0.0.0 255.0.0.0 40.0.0.2 10
Router(config)#ip route 50.0.0.0 255.0.0.0 20.0.0.2 20
Router(config)#exit
Router#show ip route static
30.0.0.0/8 is variably subnetted, 2 subnets, 2 masks
S 30.0.0.0/8 [10/0] via 20.0.0.2
S 30.0.0.100/32 [10/0] via 40.0.0.2
S 50.0.0.0/8 [10/0] via 40.0.0.2
Router#
```

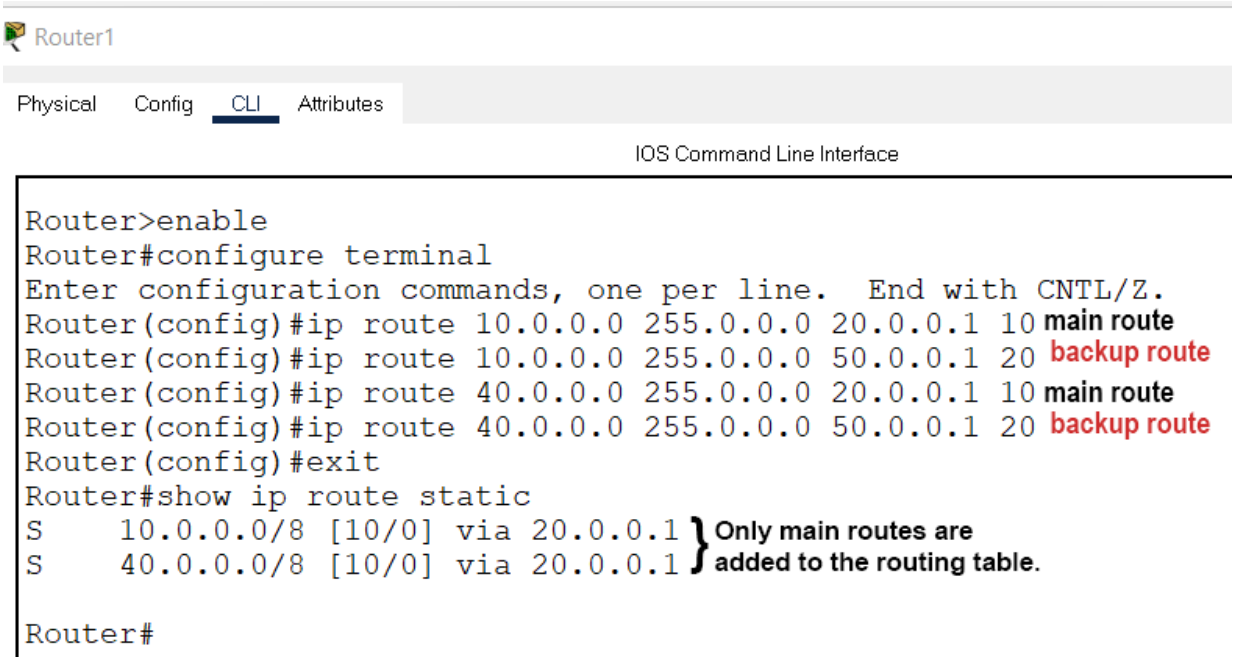


Router1 requirements

- Create two routes for network 10.0.0.0/8 and configure the first route (via -Router0) as the main route and the second route (via-Router1) as a backup route.
- Create two routes for network 40.0.0.0/8 and configure the first route (via -Router0) as the main route and the second route (via-Router2) as a backup route.
- Verify the router adds only main routes to the routing table.

Router1 configuration

```
Router>enable
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#ip route 10.0.0.0 255.0.0.0 20.0.0.1 10
Router(config)#ip route 10.0.0.0 255.0.0.0 50.0.0.1 20
Router(config)#ip route 40.0.0.0 255.0.0.0 20.0.0.1 10
Router(config)#ip route 40.0.0.0 255.0.0.0 50.0.0.1 20
Router(config)#exit
Router#show ip route static
S 10.0.0.0/8 [10/0] via 20.0.0.1
S 40.0.0.0/8 [10/0] via 20.0.0.1
Router#
```

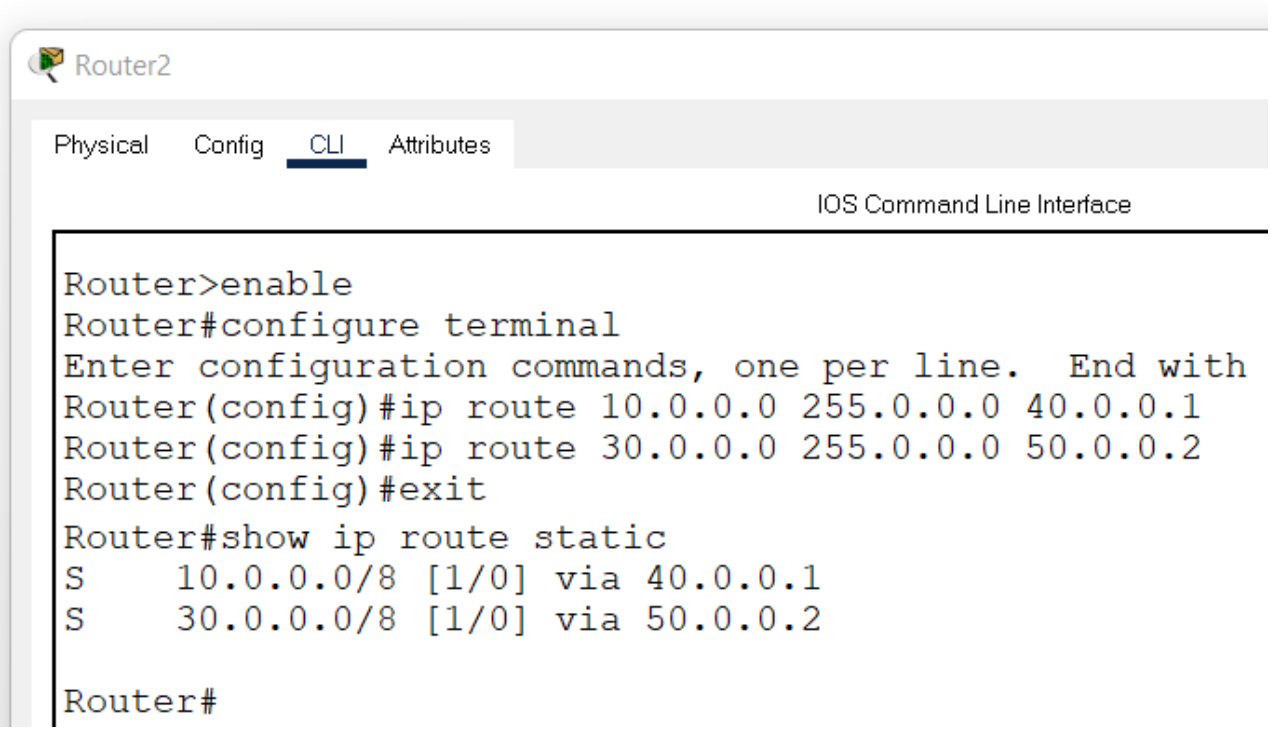


Router2 requirements

Create static routes for network 10.0.0.0/8 and network 30.0.0.0/8 and verify the router adds both routes to the routing table.

Router2 configuration

```
Router>enable
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#ip route 10.0.0.0 255.0.0.0 40.0.0.1
Router(config)#ip route 30.0.0.0 255.0.0.0 50.0.0.2
Router(config)#exit
Router#show ip route static
S 10.0.0.0/8 [1/0] via 40.0.0.1
S 30.0.0.0/8 [1/0] via 50.0.0.2
Router#
```



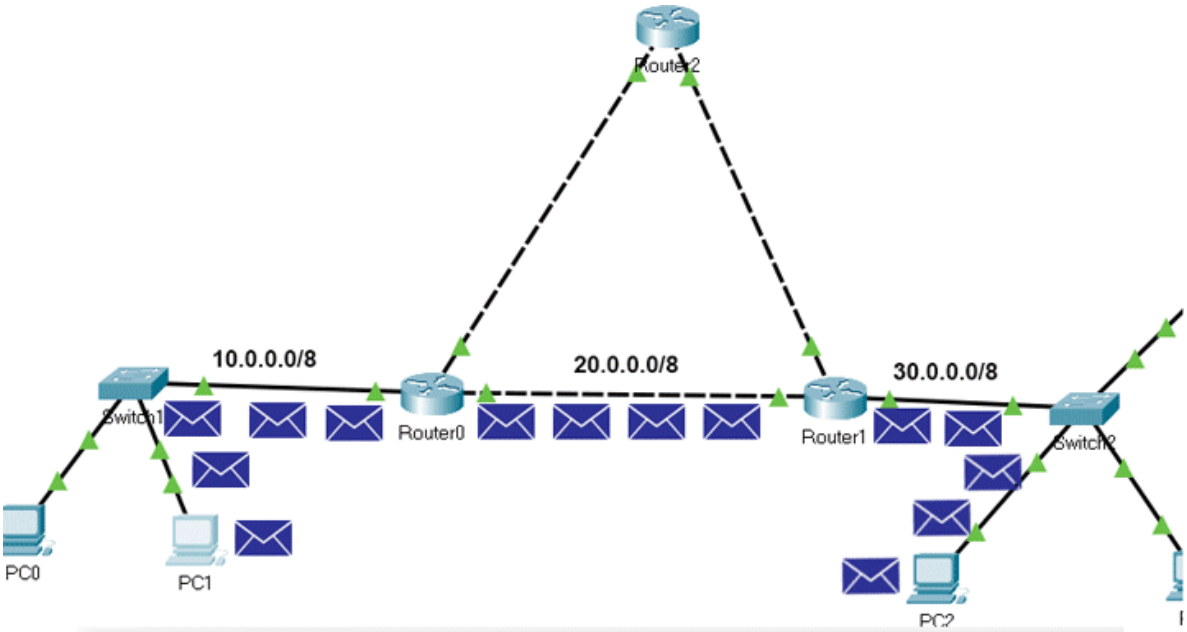
Verifying static routing

On Router0, we configured two routes for network 30.0.0.0/8. These routes are via Router1 and via Router2. We set the first route (via-Router1) as the main route and the second route as the backup route. We can verify this configuration in two ways.

By sending ping requests to a PC of network 30.0.0.0/8 and tracing the path they take to reach the network 30.0.0.0/8. For this, you can use **'tracert'** command on a PC of network 10.0.0.0/8. The '**tracert**' command sends ping requests to the destination host and tracks the path they take to reach the destination.

By listing the routing table entries on Router0. Since a router uses the routing table to forward data packets, you can check the routing table to figure out the route the router uses to forward data packets for each destination.

The following image shows the above testing.



Router0

Physical Config CLI Attributes

IOS Command Line Interface

```
Router>enable
Router#show ip route static
     30.0.0.0/8 is variably subnetted, 2 subnets, 2 masks
S       30.0.0.0/8 [10/0] via 20.0.0.2
S       30.0.0.100/32 [10/0] via 40.0.0.2
S    50.0.0.0/8 [10/0] via 40.0.0.2

Router#
```

PC1

Physical Config Desktop Programming Attributes

Command Prompt

```
C:\>tracert 30.0.0.20

Tracing route to 30.0.0.20 over a maximum of 30
hops:

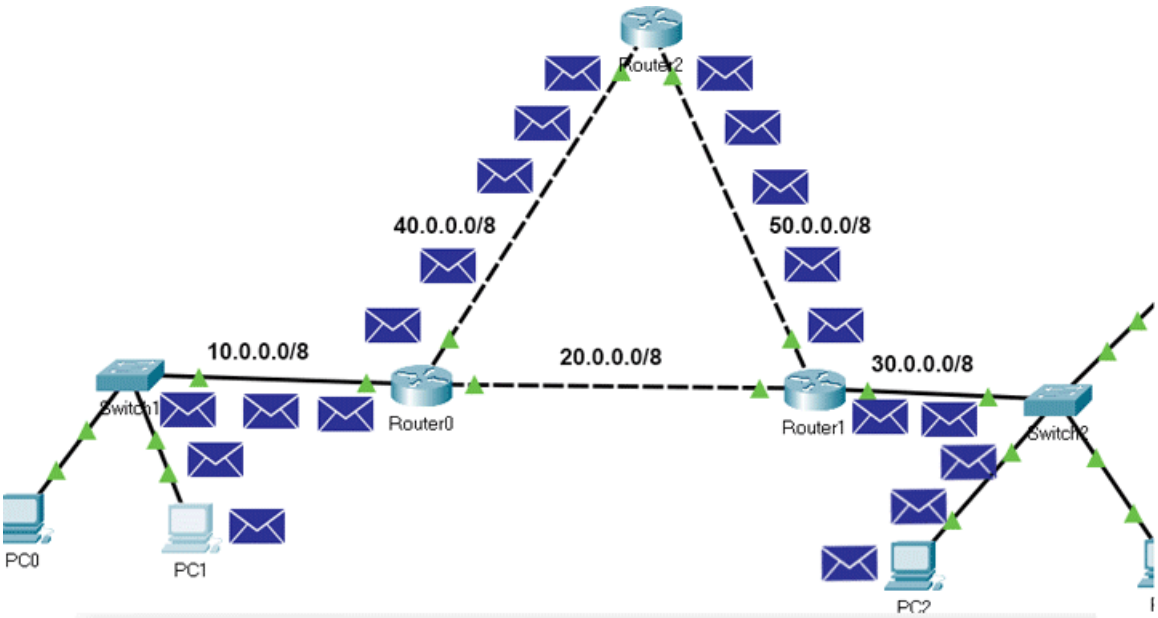
  1    0 ms      0 ms      0 ms      10.0.0.1
  2    0 ms      0 ms      0 ms      20.0.0.2
  3    *         0 ms      0 ms      30.0.0.20

Trace complete.

C:\>
```

We also configured a separate static host route for the host 30.0.0.100/8. The router must use this route to forward data packets to the host 30.0.0.100/8. To verify this, you can do the same testing for the host 30.0.0.100/8.

The following image shows this testing.



Router0

Physical Config CLI Attributes

IOS Command Line Interface

```
Router>enable
Router#show ip route static
     30.0.0.0/8 is variably subnetted, 2 subnets, 2 masks
S       30.0.0.0/8 [10/0] via 20.0.0.2
S       30.0.0.100/32 [10/0] via 40.0.0.2
S    50.0.0.0/8 [10/0] via 40.0.0.2

Router#
```

PC1

Physical Config Desktop Programming Attributes

```
Packet Tracer PC Command Line 1.0
C:\>tracert 30.0.0.100

Tracing route to 30.0.0.100 over a maximum of 30
hops:

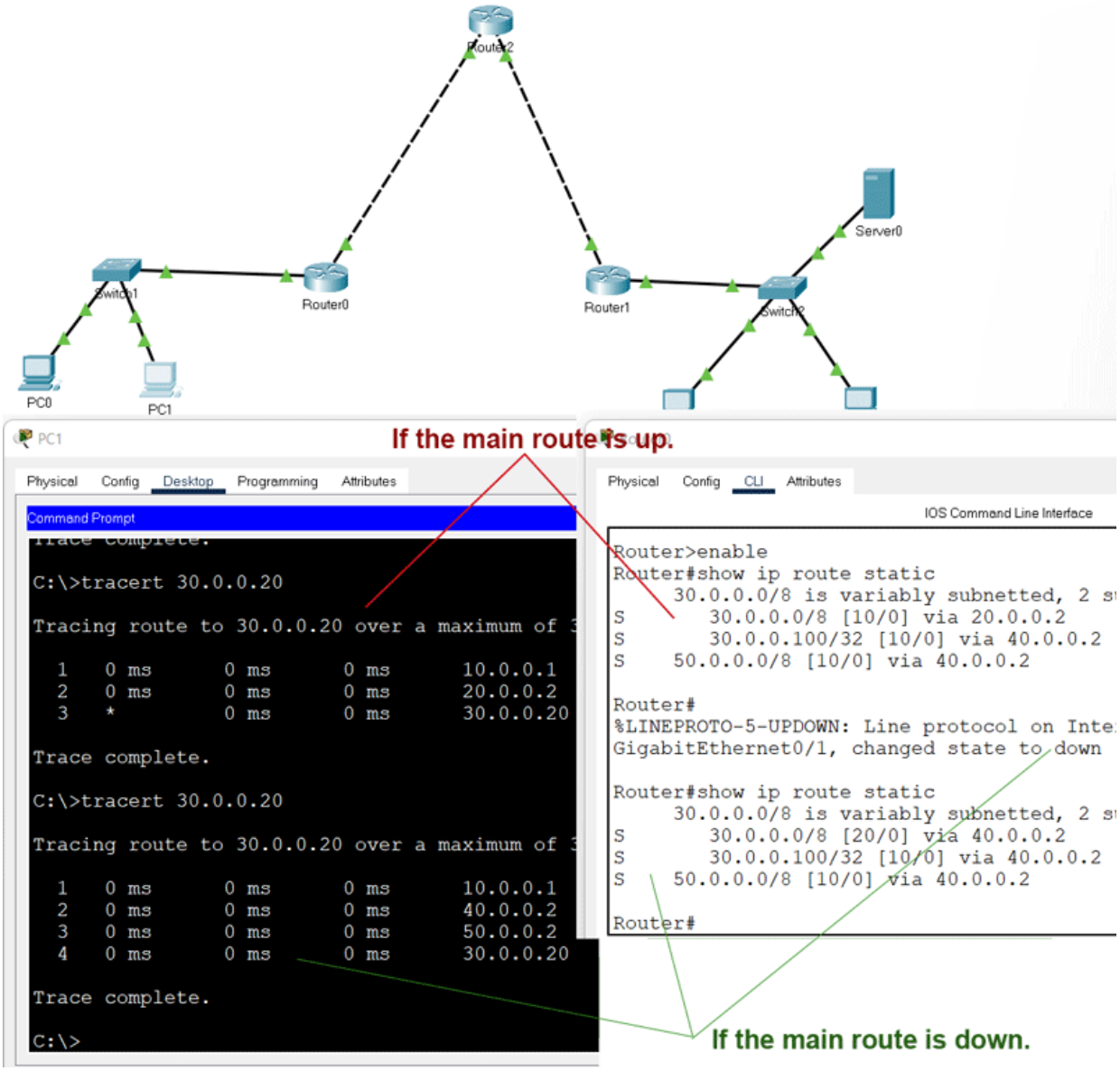
  1    1 ms       0 ms       0 ms       10.0.0.1
  2    *          0 ms       0 ms       40.0.0.2
  3    *          *          0 ms       20.0.0.2
  4    *          0 ms       0 ms       30.0.0.100

Trace complete.

C:\>
```

We also configured a backup route for network 30.0.0.0/8. The router must put the backup route to the routing table and use it to forward data packets to network 30.0.0.0/8 when the main route fails. To verify this, we have to simulate the failure of the main route.

To simulate the failure of the main route, you can delete the link between Router0 and Router1. After deleting the link, do the same testing again for the network 30.0.0.0/8.



The following link provides the configured packet tracer lab of the above example.
Packet Tracer Lab with Static Routing Configuration

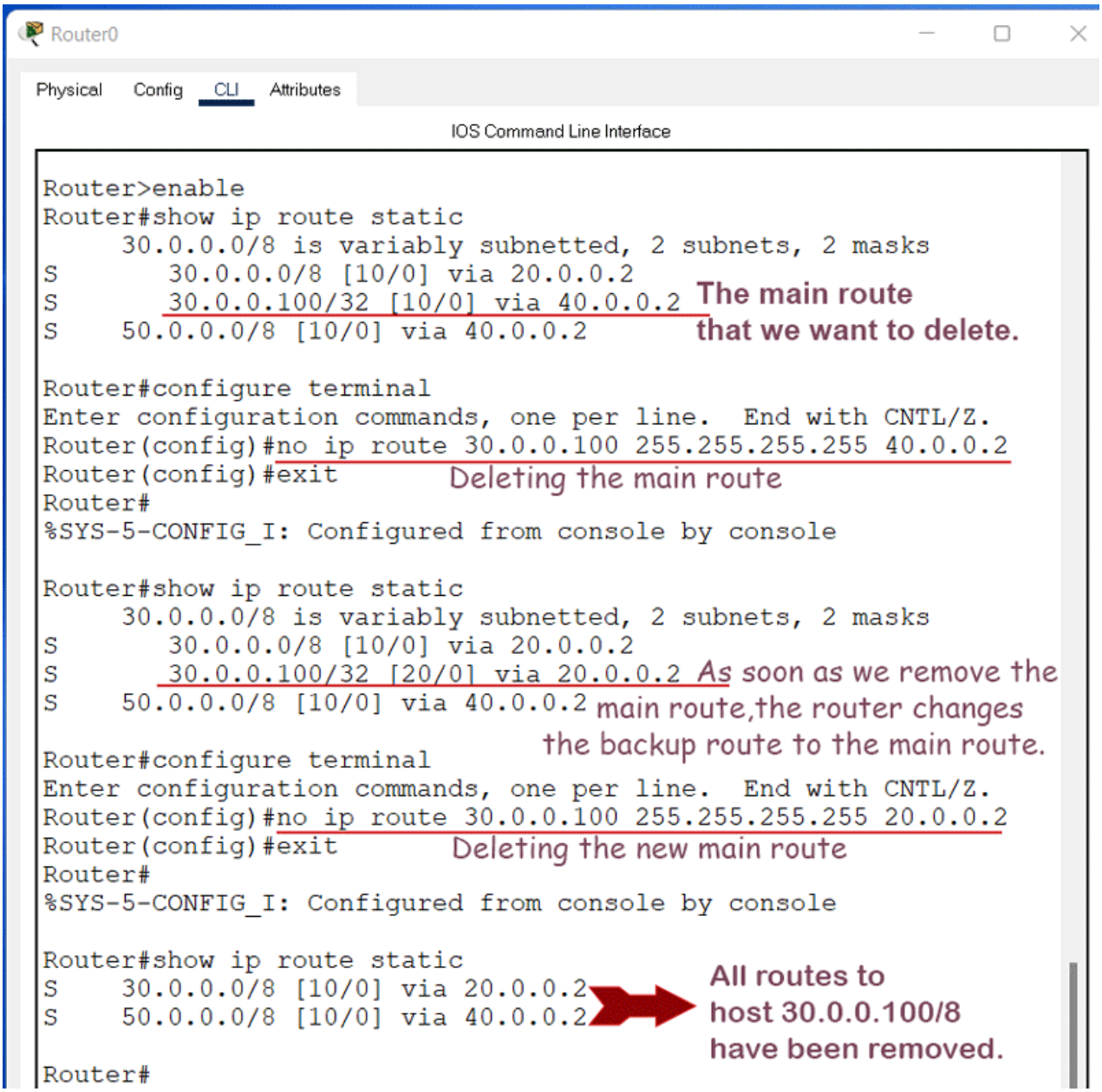
Deleting a static route

To delete a static route, use the following steps.

- Use the '**show ip route static**' command to print all static routes.
- Note down the route you want to delete.
- Use the '**no ip route**' command to delete the route.

If you have a backup route, the backup route becomes the main route when you delete the main route.

In our example, we have a backup route and a main route for the host 30.0.0.100/8. The following image shows how to delete both routes.


Router0
Physical
Config
CLI
Attributes
IOS Command Line Interface
Router>enable
Router#show ip route static
30.0.0.0/8 is variably subnetted, 2 subnets, 2 masks
S 30.0.0.0/8 [10/0] via 20.0.0.2
S 30.0.0.100/32 [10/0] via 40.0.0.2
S 50.0.0.0/8 [10/0] via 40.0.0.2
The main route that we want to delete.
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#no ip route 30.0.0.100 255.255.255.255 40.0.0.2
Router(config)#exit
Deleting the main route
Router#
%SYS-5-CONFIG_I: Configured from console by console
Router#show ip route static
30.0.0.0/8 is variably subnetted, 2 subnets, 2 masks
S 30.0.0.0/8 [10/0] via 20.0.0.2
S 30.0.0.100/32 [20/0] via 20.0.0.2
S 50.0.0.0/8 [10/0] via 40.0.0.2
As soon as we remove the main route,the router changes the backup route to the main route.
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#no ip route 30.0.0.100 255.255.255.255 20.0.0.2
Router(config)#exit
Deleting the new main route
Router#
%SYS-5-CONFIG_I: Configured from console by console
Router#show ip route static
S 30.0.0.0/8 [10/0] via 20.0.0.2
S 50.0.0.0/8 [10/0] via 40.0.0.2
All routes to host 30.0.0.100/8 have been removed.
Router#

Practical 11

AIM:- b)Simulate RIP using CISCO Packet Tracer

Initial IP configuration

Device	Interface	IP Configuration	Connected with
PC0	Fast Ethernet	10.0.0.2/8	Router0's Fa0/1
Router0	Fa0/1	10.0.0.1/8	PC0's Fast Ethernet
Router0	S0/0/1	192.168.1.254/30	Router2's S0/0/1
Router0	S0/0/0	192.168.1.249/30	Router1's S0/0/0
Router1	S0/0/0	192.168.1.250/30	Router0's S0/0/0
Router1	S0/0/1	192.168.1.246/30	Router2's S0/0/0
Router2	S0/0/0	192.168.1.245/30	Router1's S0/0/1
Router2	S0/0/1	192.168.1.253/30	Router0's S0/0/1
Router2	Fa0/1	20.0.0.1/30	PC1's Fast Ethernet
PC1	Fast Ethernet	20.0.0.2/30	Router2's Fa0/1

Assign IP address to PCs

Double click **PCs** and click **Desktop** menu item and click **IP Configuration**. Assign IP address referring the above table.

Assign IP address to interfaces of routers
Double click **Router0** and click **CLI** and press **Enter key** to access the command prompt of **Router0**.

We need to configure IP address and other parameters on interfaces before we could actually use them for routing. Interface mode is used to assign IP address and other parameters. Interface mode can be accessed from global configuration mode. Following commands are used to access the global configuration mode.

```
Router>enable
Router#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#
```

From global configuration mode we can enter in interface mode. From there we can configure the interface. Following commands will assign IP address on FastEthernet0/0.

```
Router(config)#interface fastEthernet 0/0
Router(config-if)#ip address 10.0.0.1 255.0.0.0
Router(config-if)#no shutdown
Router(config-if)#exit
Router(config)#
```

interface *fastEthernet 0/0* command is used to enter in interface mode.
ip address *10.0.0.1 255.0.0.0* command will assign IP address to interface.

no shutdown command will bring the interface up.
exit command is used to return in global configuration mode.
Serial interface needs two additional parameters **clock rate** and **bandwidth**. Every serial cable has two ends DTE and DCE. These parameters are always configured at DCE end.
We can use **show controllers interface** command from privilege mode to check the cable's end.

```
Router#show controllers serial 0/0/0
Interface Serial0/0/0
Hardware is PowerQUICC MPC860
DCE V.35, clock rate 2000000
[Output omitted]
```

Fourth line of output confirms that DCE end of serial cable is attached. If you see DTE here instead of DCE skip these parameters.
Now we have necessary information let's assign IP address to serial interface.

```
Router#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#interface serial 0/0/0
Router(config-if)#ip address 192.168.1.249 255.255.255.252
Router(config-if)#clock rate 64000
Router(config-if)#bandwidth 64
Router(config-if)#no shutdown
Router(config-if)#exit
Router(config)#interface serial 0/0/1
Router(config-if)#ip address 192.168.1.254 255.255.255.252
Router(config-if)#clock rate 64000
Router(config-if)#bandwidth 64
Router(config-if)#no shutdown
Router(config-if)#exit
Router(config)#
```

Router#configure terminal Command is used to enter in global configuration mode.
Router(config)#interface *serial 0/0/0* Command is used to enter in interface mode.
Router(config-if)#ip address *192.168.1.249 255.255.255.252* Command assigns IP address to interface. For serial link we usually use IP address from /30 subnet.
Router(config-if)#clock rate *64000* And **Router(config-if)#bandwidth *64*** In real life environment these parameters control the data flow between serial links and need to be set at service providers end. In lab environment we need not to worry about these values. We can use these values.
Router(config-if)#no shutdown Command brings interface up.
Router(config-if)#exit Command is used to return in global configuration mode.
We will use same commands to assign IP addresses on interfaces of remaining routers. We need to provided clock rate and bandwidth only on DCE side of serial interface. Following command will assign IP addresses on interface of Router1.

Router1

```
Router>enable
Router#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#interface serial 0/0/0
Router(config-if)#ip address 192.168.1.250 255.255.255.252
```

```
Router(config-if)#no shutdown
Router(config-if)#exit
Router(config)#interface serial 0/0/1
Router(config-if)#ip address 192.168.1.246 255.255.255.252
Router(config-if)#clock rate 64000
Router(config-if)#bandwidth  64
Router(config-if)#no shutdown
Router(config-if)#exit
```

Use same commands to assign IP addresses on interfaces of Router2.

Router2

```
Router>enable
Router#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#interface fastEthernet 0/0
Router(config-if)#ip address 20.0.0.1 255.0.0.0
Router(config-if)#no shutdown
Router(config-if)#exit
Router(config)#interface serial 0/0/0
Router(config-if)#ip address 192.168.1.245 255.255.255.252
Router(config-if)#no shutdown
Router(config-if)#exit
Router(config)#interface serial 0/0/1
Router(config-if)#ip address 192.168.1.253 255.255.255.252
Router(config-if)#no shutdown
Router(config-if)#exit
```

Now routers have information about the networks that they have on their own interfaces. Routers will not exchange this information between them on their own. We need to implement RIP routing protocol that will insist them to share this information.

Configure RIP routing protocol

Configuration of RIP protocol is much easier than you think. It requires only two steps to configure the RIP routing.

- Enable RIP routing protocol from global configuration mode.
- Tell RIP routing protocol which networks you want to advertise.

Let's configure it in Router0

Router0

```
Router0(config)#router rip
Router0(config-router)# network 10.0.0.0
Router0(config-router)# network 192.168.1.252
Router0(config-router)# network 192.168.1.248
```

router rip command tell router to enable the RIP routing protocol.

network command allows us to specify the networks which we want to advertise. We only need to specify the networks which are directly connected with the router.

That's all we need to configure the RIP. Follow same steps on remaining routers.

Router1

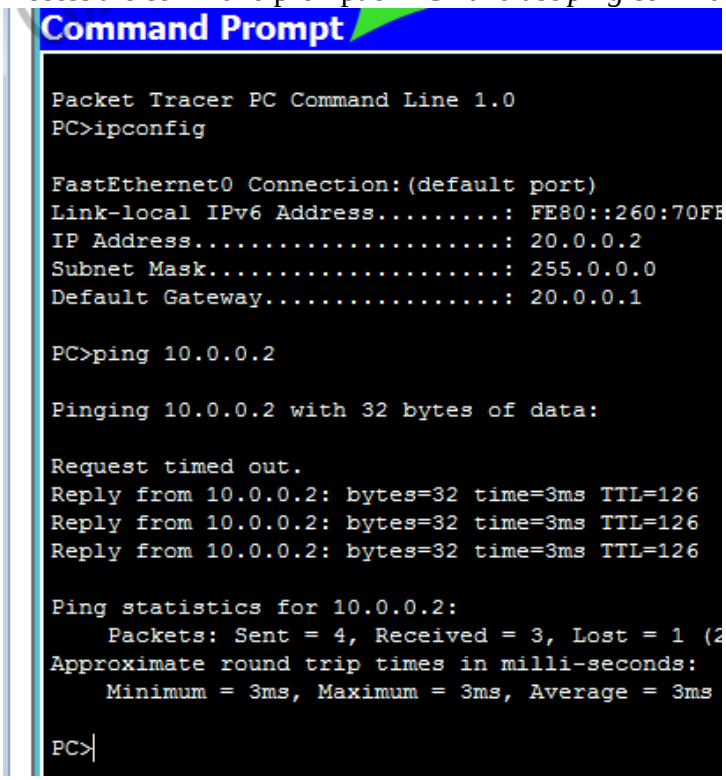
```
Router1(config)#router rip
Router1(config-router)# network 192.168.1.244
Router1(config-router)# network 192.168.1.248
```

Router2

```
Router2(config)#router rip
```

```
Router2(config-router)# network 20.0.0.0
Router2(config-router)# network 192.168.1.252
Router2(config-router)# network 192.168.1.244
```

That's it. Our network is ready to take the advantage of RIP routing. To verify the setup we will use ping command. ping command is used to test the connectivity between two devices.
Access the command prompt of **PC1** and use *ping* command to test the connectivity from **PC0**.



RIP protocol automatically manage all routes for us. If one route goes down, it automatically switches to another available. To explain this process more clearly we have added one more route in our network.
Currently there are two routes between PC0 and PC1.

Route 1

PC0 [Source / destination – 10.0.0.2] <==> Router0 [FastEthernet0/1 – 10.0.0.1] <==> Router0 [Serial0/0/1 – 192.168.1.254] <==> Router2 [Serial 0/0/1 – 192.168.1.253] <==> Router2 [FastEthernet0/0 – 20.0.0.1] <==> PC1 [Destination /source – 20.0.0.2]

Route 2

PC0 [Source / destination – 10.0.0.2] <==> Router0 [FastEthernet0/1 – 10.0.0.1] <==> Router0 [Serial0/0/0 – 192.168.1.249] <==> Router1 [Serial 0/0/0 – 192.168.1.250] <==> Router1 [Serial 0/0/1 – 192.168.1.246] <==> Router2 [Serial 0/0/0 – 192.168.1.245] <==> Router2 [FastEthernet0/0 – 20.0.0.1] <==> PC1 [Destination /source – 20.0.0.2]
By default RIP will use the route that has low hops counts between source and destination. In our network route1 has low hops counts, so it will be selected. We can use *tracert* command to verify it.
Now suppose route1 is down. We can simulate this situation by removing the cable attached between **Router0 [s0/0/1]** and **Router2 [s0/0/1]**.

What will happen now? There is no need to worry. RIP will automatically reroute the traffic.
Use ***tracert*** command again to see the magic of dynamic routing.

Practical 12

AIM: - a) Implement echo client server using TCP/UDP sockets.

Algorithm:-

Input:-

Output:-

Practical 12

AIM: - b) Implement chat client server using TCP/UDP sockets.

Algorithm:-

Input:-

Output:-

Practical 13

AIM: - Implement your own ping program

Algorithm:

Input:-

Output:-

Practical 14

AIM: - Write a code using RAW sockets to implement packet sniffing.

Algorithm:

Input:-

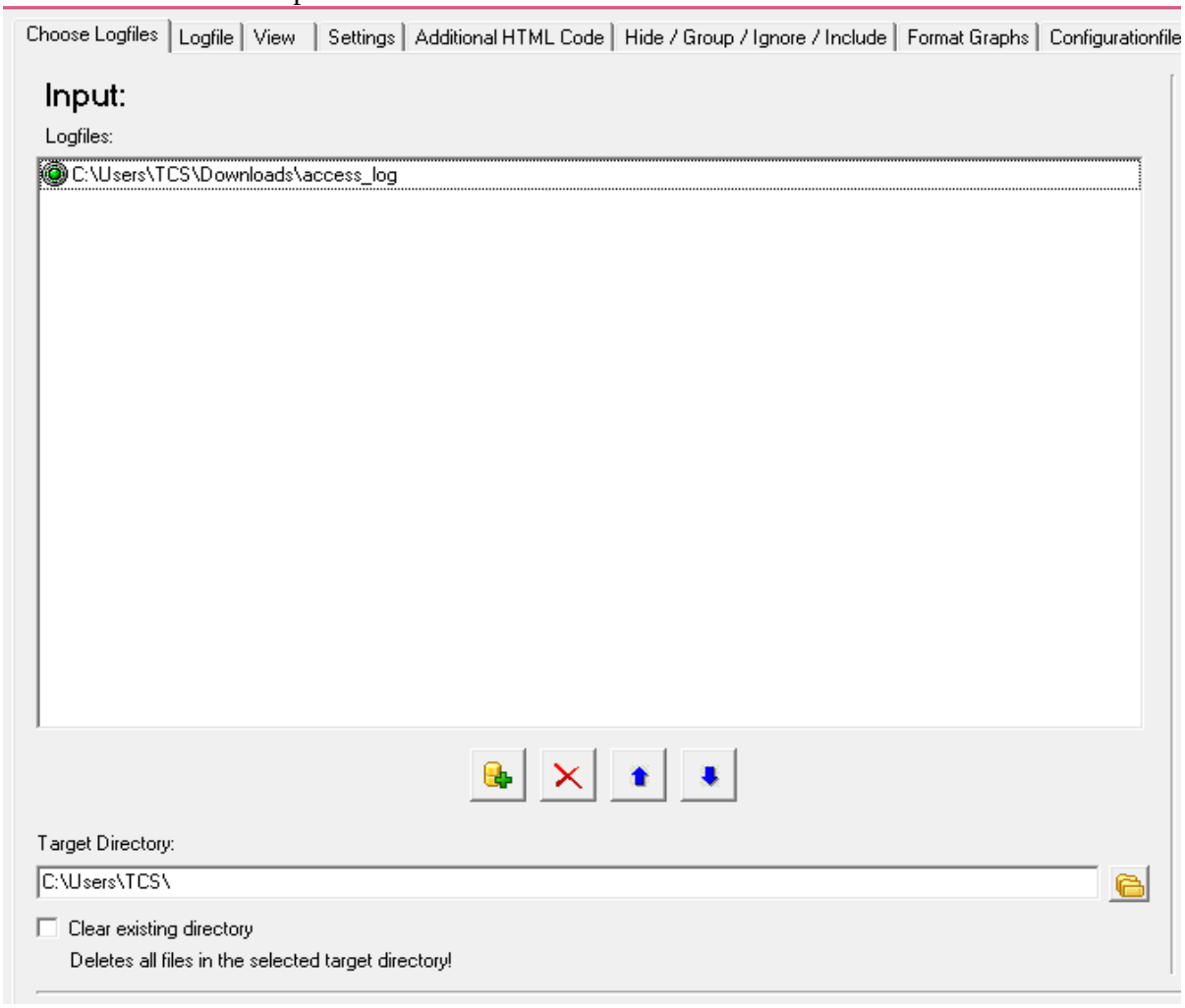
Output:-

Practical 15

AIM:- To analyze the different types of web logs using Webalizer tool.

Procedure

Step1: Run webalizer windows version
Step2. Input web log file (down load from web)
Step3: Press Run webalizer



Output:

Monthly statistics

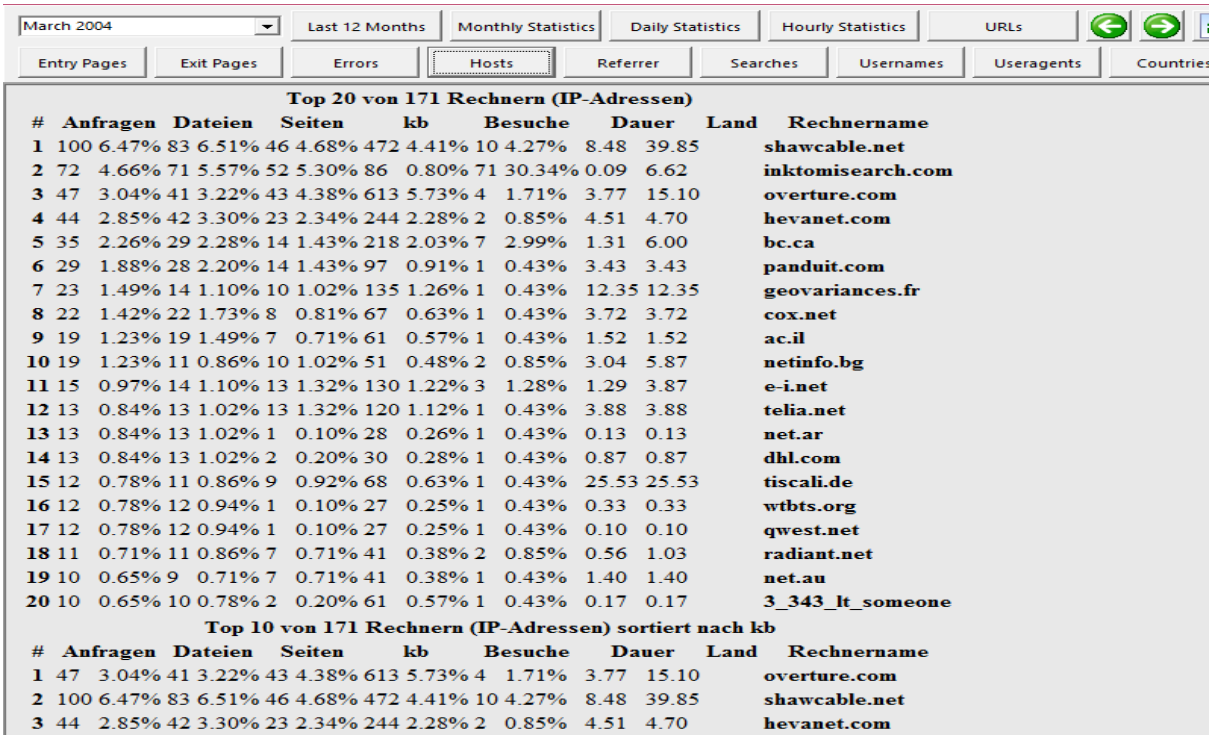


Seiten - Dokument-Abfragen (z.B. html, asp, txt); Dateien - erfolgreiche Anfragen für alle Typen (z.B. html, gif, css); Treffer - alle Anfragen, inklusive Fehler;

Tages-Statistik im Monat Maerz 2004

Tag	Anfragen			Dateien			Seiten			Besuche			Rechner			kb		
	Summe	Schnitt	Maximum	Summe	Schnitt	Maximum	Summe	Schnitt	Maximum	Summe	Schnitt	Maximum	Summe	Schnitt	Maximum	Summe	Schnitt	Maximum
7	197 12.74%	25	29	147 11.54%	18	21	186 18.94%	23	27	15 6.41%	2	3	14 8.19%	2	3	2383 22.27%	298	542
8	559 36.16%	24	63	478 37.52%	21	56	419 42.67%	18	40	57 24.36%	2	7	48 28.07%	2	7	4509 42.14%	196	386
9	232 15.01%	11	29	205 16.09%	10	29	118 12.02%	6	16	49 20.94%	2	5	44 25.73%	2	5	955 8.93%	45	138
10	286 18.50%	14	102	205 16.09%	10	34	124 12.63%	6	28	46 19.66%	2	5	38 22.22%	2	4	1336 12.49%	67	379

Hosts



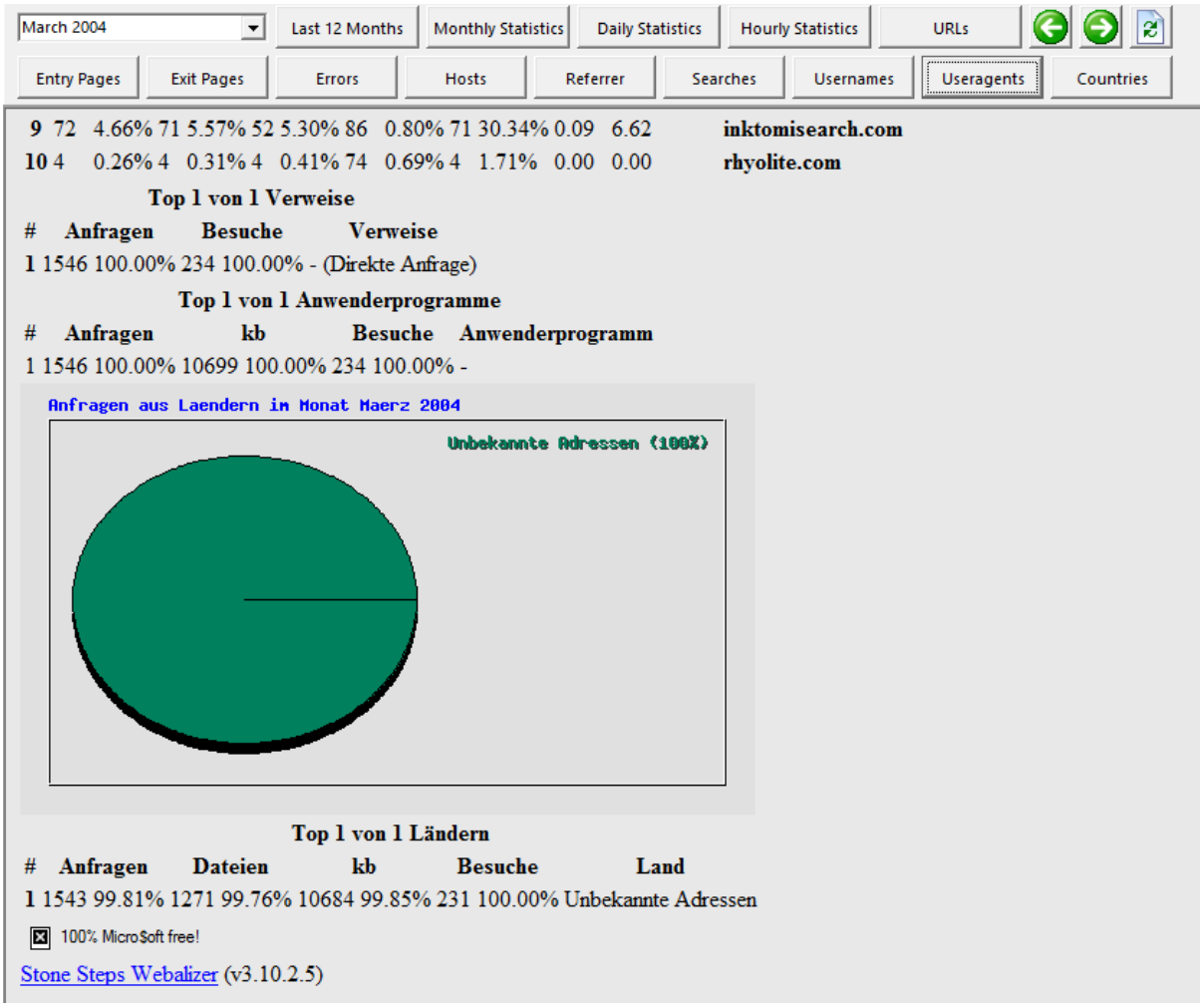
Top 20 von 171 Rechnern (IP-Adressen)

#	Anfragen	Dateien	Seiten	kb	Besuche	Dauer	Land	Rechnername
1	100 6.47%	83 6.51%	46 4.68%	472 4.41%	10 4.27%	8.48	39.85	shawcable.net
2	72 4.66%	71 5.57%	52 5.30%	86 0.80%	71 30.34%	0.09	6.62	inktomisearch.com
3	47 3.04%	41 3.22%	43 4.38%	613 5.73%	4 1.71%	3.77	15.10	overture.com
4	44 2.85%	42 3.30%	23 2.34%	244 2.28%	2 0.85%	4.51	4.70	hevanet.com
5	35 2.26%	29 2.28%	14 1.43%	218 2.03%	7 2.99%	1.31	6.00	bc.ca
6	29 1.88%	28 2.20%	14 1.43%	97 0.91%	1 0.43%	3.43	3.43	panduit.com
7	23 1.49%	14 1.10%	10 1.02%	135 1.26%	1 0.43%	12.35	12.35	geovariances.fr
8	22 1.42%	22 1.73%	8 0.81%	67 0.63%	1 0.43%	3.72	3.72	cox.net
9	19 1.23%	19 1.49%	7 0.71%	61 0.57%	1 0.43%	1.52	1.52	ac.il
10	19 1.23%	11 0.86%	10 1.02%	51 0.48%	2 0.85%	3.04	5.87	netinfo.bg
11	15 0.97%	14 1.10%	13 1.32%	130 1.22%	3 1.28%	1.29	3.87	e-i.net
12	13 0.84%	13 1.02%	13 1.32%	120 1.12%	1 0.43%	3.88	3.88	telia.net
13	13 0.84%	13 1.02%	1 0.10%	28 0.26%	1 0.43%	0.13	0.13	net.ar
14	13 0.84%	13 1.02%	2 0.20%	30 0.28%	1 0.43%	0.87	0.87	dhl.com
15	12 0.78%	11 0.86%	9 0.92%	68 0.63%	1 0.43%	25.53	25.53	tiscali.de
16	12 0.78%	12 0.94%	1 0.10%	27 0.25%	1 0.43%	0.33	0.33	wtbts.org
17	12 0.78%	12 0.94%	1 0.10%	27 0.25%	1 0.43%	0.10	0.10	qwest.net
18	11 0.71%	11 0.86%	7 0.71%	41 0.38%	2 0.85%	0.56	1.03	radiant.net
19	10 0.65%	9 0.71%	7 0.71%	41 0.38%	1 0.43%	1.40	1.40	net.au
20	10 0.65%	10 0.78%	2 0.20%	61 0.57%	1 0.43%	0.17	0.17	3_343_lt_someone

Top 10 von 171 Rechnern (IP-Adressen) sortiert nach kb

#	Anfragen	Dateien	Seiten	kb	Besuche	Dauer	Land	Rechnername
1	47 3.04%	41 3.22%	43 4.38%	613 5.73%	4 1.71%	3.77	15.10	overture.com
2	100 6.47%	83 6.51%	46 4.68%	472 4.41%	10 4.27%	8.48	39.85	shawcable.net
3	44 2.85%	42 3.30%	23 2.34%	244 2.28%	2 0.85%	4.51	4.70	hevanet.com

User-agents


March 2004
Last 12 Months
Monthly Statistics
Daily Statistics
Hourly Statistics
URLs
Entry Pages
Exit Pages
Errors
Hosts
Referrer
Searches
Usernames
Useragents
Countries
9 72 4.66% 71 5.57% 52 5.30% 86 0.80% 71 30.34% 0.09 6.62 inktomisearch.com
10 4 0.26% 4 0.31% 4 0.41% 74 0.69% 4 1.71% 0.00 0.00 rhyolite.com
Top 1 von 1 Verweise
Anfragen Besuche Verweise
1 1546 100.00% 234 100.00% - (Direkte Anfrage)
Top 1 von 1 Anwenderprogramme
Anfragen kb Besuche Anwenderprogramm
1 1546 100.00% 10699 100.00% 234 100.00% -
Anfragen aus Laendern in Monat Maerz 2004
Unbekannte Adressen (100%)
Top 1 von 1 Ländern
Anfragen Dateien kb Besuche Land
1 1543 99.81% 1271 99.76% 10684 99.85% 231 100.00% Unbekannte Adressen
100% Micro$oft free!
Stone Steps Webalizer (v3.10.2.5)